INTERACTIVE SINHALA TEXT READER FOR THE VISUALLY IMPAIRED USING OCR, NLP, AI

INDIVIDUAL THESIS

Devindi W.G.N.

IT22120670

ADVANCE DOCUMENT RETRIEVAL AND INFORMATION GENERATION

Group ID: R26-DS-002

Report submitted in partial fulfilment of the requirements for the
B.Sc. (Hons) in Information Technology
Specializing in Data Science

Department of Computing

Sri Lanka Institute of Information Technology
Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Devindi W.G.N.

Student ID: IT22120670

Signature:  Date: 11.10.2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Lakmini Abeywardhana		
Co-supervisor: Ms. Adya Dissanayaka		

Abstract

Visually impaired readers cannot check a spoken answer against the printed page, so a reading assistant must answer only from the article being read and must admit when the article does not contain the answer. Sinhala is a low-resource language, and existing retrieval-augmented generation (RAG) approaches do not adapt both the retrieved content and the wording of answers to the listener's needs. This thesis presents Component 3 of an interactive Sinhala text reader, Advance Document Retrieval and Information Generation, which aims to give faithful, personalized, speech-ready answers from Sinhala newspaper articles.

A design-science experimental approach was followed. A corpus of 500 Sinhala newspaper articles was cleaned, split into paragraph-level chunks and indexed with dense (multilingual-e5-small, Chroma DB) and BM25 retrievers combined by Reciprocal Rank Fusion. A structured personalization record controls the retrieval scope, the retrieval depth (brief, structured or full) and the prompt, and answers are generated with gpt-5.4-mini under a Sinhala-purity check. The component was evaluated offline on 72 labelled queries (56 answerable and 16 unanswerable).

The correct article appeared in the top five results for 82.1% of answerable queries, and a passage containing the key fact for 75.0%. However, the evidence passage ranked first for only 39.3%. Of the answers given, 90.2% were faithful to the retrieved text, and 15 of 16 unanswerable queries (93.8%) were correctly declined, at the cost of a 26.8% false refusal rate. Mean correctness was 1.34 out of 2, highest for summarization (1.64) and lowest for simplification (1.20). Mean Sinhala script purity was 0.986, and 95% of answers were within the word limit.

The findings show that the design keeps answers grounded, in Sinhala and suitably short for listening, while passage-level ranking remains the main weakness. The individual contribution is a style-aware hybrid RAG design, a labelled evaluation framework for Sinhala newspaper question answering, and a baseline set of measured results.

Keywords: Retrieval-Augmented Generation, Sinhala, Hybrid Retrieval, Visually Impaired Accessibility, Faithfulness

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Dr. Lakmini Abeywardhana, and my co-supervisor, Ms. Adya Dissanayaka, of the Department of Computer Science, for their guidance, encouragement and constructive feedback throughout this research. Their advice to focus my individual contribution on the parts of the system where established solutions do not yet exist shaped the direction of this work.

I am also grateful to the members of the progress-review panels, whose feedback led me to compare my work with existing approaches and to include measurable, evaluated models in my component.

I also acknowledge the creators of the openly available datasets, models and software libraries on which this work builds. In addition, I thank my group members for their collaboration on the wider Interactive Sinhala Text Reader project, and my family and friends for their constant support and patience.

Finally, I thank the Sri Lanka Institute of Information Technology for providing the academic environment in which this research was carried out.

TABLE OF CONTENTS

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 1

1.3 Problem Statement 2

1.4 Research Gap 3

1.5 Aim and Objectives 4

1.6 Research Questions or Hypotheses 5

1.7 Scope Limitations and Assumptions 6

1.8 Individual Contribution 7

1.9 Thesis Structure 7

2 LITERATURE REVIEW AND RELATED WORK 9

2.1 Review Method and Scope 9

2.2 Domain and Theoretical Foundations 9

2.2.1 Sinhala as a low-resource language 9

2.2.2 Retrieval-augmented generation 9

2.2.3 Dense, keyword-based and fused retrieval 10

2.3 Related Research 10

2.3.1 Evaluating faithfulness and abstention in RAG 10

2.3.2 Use of retrieved context by language models 11

2.4 Existing Systems Products and Techniques 11

2.4.1 Reading aids for printed news 11

2.4.2 Retrieval and generation techniques 11

2.5 Gap Synthesis and Conceptual Framework 12

2.5.1 Gap synthesis 12

2.5.2 Conceptual framework 12
3 RESEARCH METHODOLOGY 13
3.1 Research Design 13
3.1.1 Research approach and rationale 13
3.1.2 Research questions 13
3.1.3 Study setting and conceptual framework 14
3.2 Data Participants or Study Materials 15
3.2.1 Target population and dataset description 15
3.2.2 Sampling and eligibility criteria 15
3.2.3 Instruments, datasets and study materials 16
3.3 Experimental or Study Procedure 18
3.3.1 Data-collection and experimental process 18
3.3.2 Data management and quality control 18
3.4 Measures and Evaluation Metrics 19
3.5 Data Analysis 21
3.5.1 Data sources and analysis plan 21
3.5.2 Corpus profiling 22
3.5.3 Consolidation of evaluation data 22
3.5.4 Retrieval analysis (RQ1) 23
3.5.5 Faithfulness and abstention analysis (RQ2) 23
3.5.6 Correctness analysis by intent type (RQ3) 24
3.5.7 Language quality and format analysis (RQ4) 24
3.6 Ethics Privacy Security and Responsible AI 24
3.6.1 Ethical approval and informed consent 24
3.6.2 Privacy, security and data governance 24
3.7 Reproducibility and Research Artefacts 25

3.7.1 Data, code and materials availability 25
3.7.2 Documentation, versioning and computational environment 26
3.7.3 Limitations on reproducibility 27
4 PRODUCT AND SOLUTION DESIGN 28
4.1 Requirements 28
4.1.1 Stakeholders and user needs 28
4.1.2 Functional requirements 29
4.1.3 Non-functional requirements 30
4.1.4 Constraints and assumptions 31
4.2 Component Architecture within the Complete Solution 32
4.2.1 Overall solution architecture 32
4.2.2 Internal architecture of Component 3 33
4.2.3 Data flow and system interactions 35
4.2.4 Architecture rationale 37
4.3 Design Alternatives and Decisions 38
4.3.1 Alternatives considered 38
4.3.2 Selected design and justification 39
4.4 Detailed Design 40
4.4.1 User-interface and user-flow design 40
4.4.2 Use cases and process flows 41
4.4.3 Database and data-model design 43
4.5 Standards and Professional Practice 47
5 IMPLEMENTATION INTEGRATION AND TESTING 49
5.1 Development Environment and Technology Stack 49
5.1.1 Development environment 49
5.1.2 Technology stack 49

5.1.3 Dependencies, licences and configuration 49
5.1.4 Development process and source-code management 50
5.2 Component Implementation 50
5.2.1 Implementation overview 50
5.2.2 Data pipeline and storage implementation 51
5.2.3 Algorithm and business-logic implementation 52
5.2.4 Security and privacy implementation 53
5.3 Integration with the Other Components 54
5.3.1 User-interface implementation 54
5.4 Verification and Software Testing 54
5.4.1 Verification and testing strategy 54
5.4.2 Test environment and test data 55
5.4.3 Module-level verification 56
5.4.4 Interface and integration testing 58
5.4.5 Security and privacy verification 58
5.5 Deployment and Operation 59
5.5.1 Deployment model and target environment 59
5.5.2 Installation and configuration procedure 60
5.5.3 Runtime operation 61
5.5.4 Configuration, secrets and version management 62
5.5.5 Deployment limitations and requirements for production use 62
6 RESULTS AND VALIDATION 64
6.1 Research Results 64
6.1.1 Retrieval (RQ1) 64
6.1.2 Faithfulness and abstention (RQ2) 64
6.1.3 Correctness by intent type (RQ3) 65

6.1.4 Language quality and format (RQ4) 65
6.2 Product and Technical Results 65
6.3 User or Stakeholder Validation 66
6.4 Integration Results 66
6.5 Summary of Findings 67
7 DISCUSSION 68
7.1 Interpretation of Findings 68
7.2 Comparison with Literature Baselines and Existing Solutions 69
7.3 Research Contribution 71
7.4 Product Contribution 71
7.5 Limitations and Threats to Validity 72
7.6 Implications 73
8 COMMERCIALIZATION AND SUSTAINABILITY 74
8.1 Value Proposition and Target Users 74
8.1.1 Target users 74
8.1.2 Value proposition 74
8.2 Market and Competitive Position 74
8.2.1 Alternatives available to users 74
8.2.2 Competitive position 75
8.3 Business or Sustainability Model 75
8.3.1 Possible models 75
8.3.2 Costs and sustainability 76
8.4 Scalability, Intellectual Property and Compliance 76
8.4.1 Scalability 76
8.4.2 Intellectual property 76
8.4.3 Compliance 77

8.5 Contribution to the Sustainable Development Goals 77
9 CONCLUSION AND FUTURE DIRECTIONS 78
9.1 Conclusion 78
9.2 Achievement of Objectives 79
9.3 Individual Research and Product Contributions 80
9.4 Future Directions 81
REFERENCES 83
GLOSSARY 85
APPENDICES 87
Appendix A Research Instruments or Extended Results 87
Appendix B Academic Integrity and AI Use Disclosure 88

LIST OF FIGURES

Figure 4.1: Overall solution architecture 33

Figure 4.2: Component 3 RAG system architecture 38

LIST OF TABLES

Table 2.1: Retrieval and generation techniques reviewed 11

Table 3.1: Study materials by component stage 16

Table 3.2: Mapping of research questions to analysis 21

Table 3.3: Data classification and handling 24

Table 3.4: Availability of research artefacts 25

Table 4.1: Stakeholders and their main needs 28

Table 4.2: Functional requirements 29

Table 4.3: Non-functional requirements 30

Table 4.4: Responsibilities of the four components 32

Table 4.5: Modules of Component 3 34

Table 4.6: Design decisions and alternatives considered 38

Table 4.7: Design principles for spoken answers 40

Table 4.8: Use cases of Component 3 41

Table 4.9: Article record after cleaning 43

Table 4.10: Chunk record 44

Table 4.11: Input record 44

Table 4.12: Output record 45

Table 4.13: Exception and error handling 46

Table 5.1: Technology stack 49

Table 5.2: Priority rules of the request resolver 52

Table 5.3: Adaptive retrieval depth by detail_level 52

Table 5.4: Levels of verification 55

Table 5.5: Properties verified at module level 56

Table 5.6: Deployed elements 60

Table 5.7: Installation and configuration procedure 60

Table 6.1: Retrieval accuracy at article level and evidence level 64

Table 6.2: Mean correctness by intent type 65

Table 6.3: Language quality and format results 65

Table 6.4: Required behaviors of Component 3 and supporting evidence 66

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
BM25	Best Matching 25 (keyword-based ranking function)
CPU	Central Processing Unit
FR	Functional Requirement
GB	Gigabyte
GHz	Gigahertz
GPU	Graphics Processing Unit
ID	Identifier
IEC	International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MRR	Mean Reciprocal Rank
NFC	Normalization Form C (Unicode)
NFR	Non-Functional Requirement
NIST	National Institute of Standards and Technology
NLP	Natural Language Processing
OCR	Optical Character Recognition
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory

Abbreviation	Meaning
RMF	Risk Management Framework
RQ	Research Question
RRF	Reciprocal Rank Fusion
SDG	Sustainable Development Goal
SHA	Secure Hash Algorithm
UC	Use Case
UN	United Nations
UTF	Unicode Transformation Format
WCAG	Web Content Accessibility Guidelines
ZWJ	Zero-Width Joiner

1 INTRODUCTION

1.1 Overall Project Context

Printed newspapers remain an important source of local news, yet they are closed to readers who cannot see the page. More than 40 million people worldwide were estimated to be blind in 2020, and many more lived with moderate or severe vision impairment [1]. Screen-reading software helps only when text is already digital, so printed pages stay out of reach. A reading aid for these users must therefore start from the printed page and end in speech.

This thesis is part of the group project R26-DS-002, an interactive Sinhala text reader that carries a reader from a printed newspaper page to a spoken answer. The complete solution is a pipeline of four components. Component 1 reads the newspaper image and produces Sinhala text by optical character recognition (OCR). Component 2 corrects the OCR errors and passes the corrected text onward. Component 3, the subject of this thesis, retrieves the relevant article content and generates a personalized Sinhala answer. Component 4 handles voice interaction: it receives the listener's spoken request, prepares a structured personalization record that describes what the listener wants, and reads the final answer aloud. The components exchange only defined JSON records, so each can be tested and replaced separately.

Sinhala makes every stage of this pipeline harder. Sinhala has far fewer labelled datasets, ready-made models and evaluation sets than English, and it is commonly described as a low-resource language [2], [3]. Work on Sinhala language technology has also concentrated on foundational tasks, such as the comparison of pre-trained language models for text classification [4].

1.2 Individual Component Context

Component 3 is titled Advance Document Retrieval and Information Generation. It sits between the text-correction stage and the voice stage. Its inputs are the corrected article text from Component 2 and the personalization record from Component 4. The record carries the intent of the request (for example summarize, explain,

simplify or elaborate), the wording style the listener prefers, a prompt modifier and optional flags such as the level of detail. Its output is a short Sinhala answer, together with the identifiers of the article and passages used, which Component 4 then converts to speech.

The component is built on retrieval-augmented generation (RAG). In a RAG system, a retriever first selects passages from an external knowledge source, and a language model then generates its answer from the question and those passages [5]. Recent surveys describe RAG as a family of designs that differ in how retrieval, augmentation and generation are combined [6]. It suits this project because answers must come from the article being read and not from the general knowledge of the model, which can produce fluent but unsupported statements [7].

In Component 3, a corpus of 500 Sinhala newspaper articles is cleaned and divided into paragraph-level chunks. Two indexes are built over these chunks. The first is a dense index that uses the multilingual-e5-small embedding model [8] stored in ChromaDB, which retrieves passages by meaning in the manner of dense passage retrieval [9]. The second is a BM25 index, a keyword-based ranking function that rewards exact matches such as names, places and numbers [10]. The two ranked lists are merged by Reciprocal Rank Fusion (RRF), which uses only rank positions and therefore avoids the need to compare scores that lie on different scales [11]. The personalization record controls how much of the article is retrieved (brief, structured or full depth) and how the prompt is worded. The gpt-5.4-mini language model generates the answer, which is checked for Sinhala script purity.

1.3 Problem Statement

A listener who cannot see the page cannot compare a spoken answer with the printed text, so an invented name, date or figure would go unnoticed. Faithfulness to the source article is therefore the first requirement of the component, ahead of fluency or style. Language models are known to generate content that is fluent but unsupported by the source, and this behavior, called hallucination, has been documented across many generation tasks [7].

Retrieval reduces this risk but does not remove it. Evaluation of language models in RAG settings has shown that, even when relevant documents are supplied, models still struggle to reject a question when the supplied documents do not contain the answer [12]. The way retrieved text is placed in the prompt also matters, because models can use relevant information less reliably when it is buried within a long context [13]. In news reading, a retriever may rank a neighboring paragraph above the one that holds the answer, and the model may then answer from the wrong passage or refuse a question that the article can answer.

Listening adds three further needs. Listeners differ, so one fixed style does not suit everyone: some want a short direct reply, others a simplified or step-by-step explanation. Spoken answers must be short because the listener cannot skim ahead. The answer must also be in Sinhala, since mixed-language output is confusing when read aloud.

The problem addressed by this thesis is therefore stated as follows. **How can a Sinhala reading assistant generate answers from newspaper articles that are faithful to the article, able to decline questions the article cannot answer, adapted to the listener's requested intent, style and depth, and suitable for listening, when it must work with limited Sinhala language resources and a noisy newspaper corpus?**

1.4 Research Gap

The gap identified by this thesis is found where four lines of work meet: RAG, hybrid retrieval, personalized answer generation and Sinhala language technology. Each line is well developed on its own, but their combination has received little attention in the sources reviewed. Section 2 gives full review.

Language coverage. Prominent RAG benchmarks have been built for English and Chinese [12]. For Sinhala, the resource gap described in the language-technology literature [2], [3] means that evaluation data for question answering over news is scarce. To the best of the author's knowledge, a labelled evaluation of faithfulness and refusal behavior for Sinhala newspaper question answering was not found in the sources reviewed.

Retrieval for an inflected language. Dense retrieval can outperform keyword matching on many question answering tasks [9], whereas keyword ranking such as BM25 remains strong for exact terms [10]. Rank fusion is a simple, established way to combine the two [11]. In Sinhala, however, one word appears in many inflected forms, which weakens exact keyword matching, and the behavior of the combined approach on Sinhala news was not found in the sources reviewed. This thesis builds a hybrid retriever for Sinhala news and reports how it performs, but it does not compare it with dense-only or BM25-only retrieval, so whether the hybrid approach works better for Sinhala remains open and is left to future work.

Control by the listener's needs. RAG surveys describe how retrieval, augmentation and generation can be improved [6], but in the sources reviewed, the amount of article content retrieved and the wording of the answer are generally not driven by a structured record of what the listener asked for. For a listener who depends on speech, both the content and the form of the answer need to follow the request. This thesis builds such a record-driven control and measures correctness by request type, but it does not compare it with prompt-only personalization, which is left to future work.

Evaluation for listening. Existing evaluations measure abilities such as noise robustness and negative rejection [12], but they do not combine these with script purity and length suited to listening, which matter for spoken Sinhala.

The research gap is therefore the absence of a style-aware hybrid RAG design for Sinhala newspaper reading, together with a labelled evaluation framework that measures it against the needs of listeners who cannot check the printed page. This thesis identifies the gap, builds a design and an evaluation framework for it, and reports the measured behavior of that design. Comparison with alternative designs is left to future work.

1.5 Aim and Objectives

Aim. The aim of this study is to design, implement and evaluate a style-aware hybrid retrieval-augmented generation component for an interactive Sinhala text reader, so

that it gives faithful, personalized and speech-ready answers from Sinhala newspaper articles.

To achieve this aim, the following objectives were set.

O1. Design a style-aware RAG component that works with the other components of the reading system through a fixed JSON contract.

O2. Build a hybrid retrieval pipeline over a cleaned Sinhala newspaper corpus, combining dense and BM25 search through Reciprocal Rank Fusion on paragraph-level chunks.

O3. Produce answers that are grounded in the article and decline questions that the knowledge base cannot answer.

O4. Produce speech-ready answers in Sinhala, with a length suited to listening.

O5. Support different request types (direct answer, summarization, simplification and elaboration) and wording styles through the personalization record.

1.6 Research Questions or Hypotheses

The study is guided by four research questions. They are answered by the offline evaluation described in Section 3 and reported in Section 6.

RQ1. Retrieval. How accurately does the retriever identify the correct source article and the passage containing the key fact for Sinhala newspaper questions?

RQ2. Faithfulness and abstention. How faithfully are the generated answers to the retrieved newspaper content, and how reliably does the system decline questions that the knowledge base cannot answer?

RQ3. Correctness by intent type. How does answer correctness vary across the requested intent types (direct answer, summarization, simplification and elaboration)?

RQ4. Language quality and format. To what extent do the generated answers remain in pure Sinhala, stay within the word limits and read fluently?

These are descriptive questions and not formal hypotheses, because the evaluation measures one design without comparing it with alternatives. RQ1 relates to objective O2, RQ2 to O3, RQ3 to O5 and RQ4 to O4.

1.7 Scope Limitations and Assumptions

Scope. The study covers Component 3 only. It includes the cleaning and paragraph-level chunking of a corpus of 500 Sinhala newspaper articles, hybrid dense and BM25 retrieval with Reciprocal Rank Fusion, depth-controlled retrieval driven by the personalization record, answer generation with a hosted language model, and the Sinhala-purity check. The component was evaluated offline on 72 labelled queries, of which 56 can be answered from the corpus and 16 cannot.

Out of scope. The following are outside this study:

- OCR (Component 1) and text correction (Component 2), whose output is assumed.
- Speech recognition, speech synthesis and the production of the personalization record (Component 4).
- Training or fine-tuning of language models, because the design relies on prompt-based control with a hosted model.
- User studies with listeners and a controlled measurement of response time.
- A comparison with alternative designs, such as dense-only retrieval or prompt-only personalization.

Limitations. Faithfulness and correctness were scored with a language-model judge, which can be inconsistent, and each query was run once on a hosted model. With 56 answerable queries, one query equals about 1.8 percentage points, so differences between intent types are indicative only. The corpus and test set are small, so results may not be transferred to other newspapers or larger collections (see Section 7.5).

Assumptions. The study assumes that Component 2 delivers corrected Sinhala text of acceptable quality, that the personalization record from Component 4 is valid, that the listener reads one article at a time and the system knows which one is active, and that the hosted language model follows Sinhala instructions reliably.

1.8 Individual Contribution

This thesis reports on the individual work of the author on Component 3. The research contributions are as follows.

- **A style-aware hybrid RAG design for Sinhala newspaper reading.** A structured control record adjusts both the prompt and the amount of article content retrieved, and dense and BM25 retrieval are combined through Reciprocal Rank Fusion on paragraph-level chunks.
- **An evaluation framework for Sinhala newspaper question answering.** It consists of a labelled test set of 72 queries, including two kinds of unanswerable queries (off-topic and near-topic), and measures that cover retrieval, faithfulness, abstention, correctness by intent and language format.
- **A baseline set of measured results** for this design, which later work can use for comparison.

The product contributions are working Component 3 that accepts the personalization record, answers from the article text, returns speech-ready Sinhala and reports the passages used so that every answer can be traced. The design is modular, needs no model training, and runs on ordinary hardware with a hosted language model. A reusable evaluation runner repeats the same measurements whenever the component changes.

These contributions describe the design and its measured behavior. Because the evaluation did not compare the design with alternatives, the thesis does not claim that it outperforms them.

1.9 Thesis Structure

The remainder of this thesis is organized as follows.

Section 2 reviews the related literature and synthesizes the research gap. **Section 3** describes the research methodology, including the design, the data, the procedure and the evaluation measures. **Section 4** presents the requirements, the architecture and the detailed design of the component. **Section 5** describes implementation, integration and testing. **Section 6** reports the research and technical results. **Section 7**

interprets the findings and discusses limitations. **Section 8** considers commercialization and sustainability. **Section 9** concludes the thesis, assesses the achievement of the objectives and suggests future directions. The References, Glossary and Appendices follow Section 9.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

This chapter is a focused narrative review, not a systematic review. It covers the four lines of work that meet in Component 3: language technology for low-resource languages, especially Sinhala; retrieval-augmented generation (RAG) and the hallucination problem it is meant to reduce; dense, keyword-based and fused retrieval; and benchmarks that test how language models behave in RAG settings. It draws on the 13 sources in the References: eleven peer-reviewed papers and two arXiv documents [6], [8].

A source was included when it did at least one of three things: it introduced or surveyed a method used in the component, it gave evidence about language-model behavior that affects the design, or it described the position of Sinhala in language technology. Because the review is not exhaustive, a gap means only that no such work was found in the sources reviewed.

2.2 Domain and Theoretical Foundations

2.2.1 Sinhala as a low-resource language

Joshi et al. [2] grouped the world's languages by the amount of labelled and unlabeled data available for them and showed that the benefits of natural language processing (NLP) are spread very unevenly, with most languages receiving little attention. Ranathunga and de Silva [3] probed this disparity more deeply. Together, these two studies are the basis on which this thesis describes Sinhala as a low-resource language. Dhananjaya et al. [4] compared pre-trained language models for Sinhala text classification, a foundational task; none of the Sinhala studies reviewed reports retrieval or answer generation for news.

2.2.2 Retrieval-augmented generation

Lewis et al. [5] introduced RAG as a model that combines a pre-trained generator with a non-parametric memory, a dense index of documents reached through a neural retriever. Gao et al. [6] survey the field and organize it into Naive, Advanced and

Modular RAG, which differ in how retrieval, augmentation and generation are combined.

The reason for grounding is hallucination. Ji et al. [7] survey hallucination in natural language generation and separate content that contradicts the source from content that cannot be checked against it. For a listener who cannot check the printed page, both kinds are harmful.

2.2.3 Dense, keyword-based and fused retrieval

BM25 is a keyword-based ranking function from the probabilistic relevance framework [10]. It rewards exact term matches, so it suits names, places and numbers, but depends on the query words appearing in the passage. Dense passage retrieval [9] encodes questions and passages into vectors with a dual encoder and ranks passages by similarity of meaning. Karpukhin et al. reports that it outperformed a strong BM25 system by 9% to 19% absolute in top 20 passage accuracy on open-domain question answering [9]. Multilingual E5 [8] provides such embeddings for many languages; its usefulness for Sinhala news is tested here, not assumed.

Reciprocal Rank Fusion (RRF) combines several ranked lists using only the ranked positions of documents. A document d receives the score in (1), where R is the set of ranked lists, r(d) is the rank of d in a list and k is a smoothing constant (Cormack et al. used k = 60) [11]. Cormack et al. report that RRF outperformed Condorcet fusion and individual rank-learning methods [11]. Because it ignores raw scores, it can combine BM25 and dense similarities, which lie on different scales.

$$RRF(d) = \Sigma_{r \in R} \; 1 / (k + r(d)) \tag{1}$$

2.3 Related Research

2.3.1 Evaluating faithfulness and abstention in RAG

Chen et al. [12] built a benchmark for RAG that tests four abilities of a language model: noise robustness, negative rejection, information integration and counterfactual robustness. The benchmark covers English and Chinese and shows clear model weaknesses in these abilities. Negative rejection, declining a question

that the supplied documents cannot answer, is the most relevant here and corresponds to the abstention measure of RQ2.

2.3.2 Use of retrieved context by language models

Liu et al. [13] studied how language models use long input contexts and found that performance is highest when the relevant information is at the beginning or the end of the input and falls when it is in the middle. The amount and order of retrieved text therefore affect answer quality, which supports controlling how much article content is retrieved (brief, structured or full depth).

2.4 Existing Systems Products and Techniques

2.4.1 Reading aids for printed news

Screen-reading software helps only when text is already digital, so printed pages remain closed to readers who cannot see them [1]. The reviewed sources do not describe existing reading aids for printed Sinhala newspapers, so no product-level comparison is made; the techniques from which such a system can be built are compared instead.

2.4.2 Retrieval and generation techniques

Table 2.1 summarizes the techniques reviewed and the limitation of each that matters for Component 3.

Table 2.1: Retrieval and generation techniques reviewed

Technique	Strength	Limitation for Component 3	Source
Keyword ranking (BM25)	Exact matches such as names, places and numbers	Depends on query words appearing in the passage	[10]
Dense retrieval	Matches by meaning; outperformed BM25 on open-domain question answering	Quality for Sinhala news depends on the embedding model and must be tested	[8], [9]
Rank fusion (RRF)	Combines ranked lists using ranks only, without comparing scores	Depends on the quality of the lists that are fused	[11]
RAG generation	Grounds the answer in retrieved text	Models can still hallucinate or fail to reject unanswerable questions	[5]-[7], [12]
Context placement	Order and amount of context can be tuned	Relevant text in the middle of a long context is used less reliably	[13]

2.5 Gap Synthesis and Conceptual Framework

2.5.1 Gap synthesis

The review shows that each technique needed by Component 3 is established on its own: RAG [5], [6], dense and keyword retrieval [9], [10], rank fusion [11], and benchmarks of faithfulness and rejection [12]. Four gaps remain in the sources reviewed, and they are the gaps stated in Section 1.4.

1. **Language coverage.** The RAG benchmark [12] covers English and Chinese, and the Sinhala literature [2]-[4] shows limited resources and a focus on classification. A labelled evaluation of faithfulness and refusal for Sinhala newspaper question answering was not found.
2. **Retrieval for an inflected language.** Dense and keyword retrieval each have strengths [9], [10] and RRF is a simple way to combine them [11], but their combined behavior on Sinhala news was not found.
3. **Control by the listener's needs.** RAG surveys describe how retrieval, augmentation and generation can be improved [6], but a structured record of the listener's request was not found driving the amount retrieved and the wording of the answer.
4. **Evaluation for listening.** Existing evaluation covers noise robustness and negative rejection [12], but not script purity or answer length for spoken Sinhala.

2.5.2 Conceptual framework

The framework links each risk found in the literature to a design element of Component 3 and to a research question. A personalization record fixes the intent, style and depth of the request. Depth-controlled hybrid retrieval combines dense search [8], [9] and BM25 [10] through RRF [11], and the retrieved amount is adjusted to limit the loss of information in long contexts [13]. The model answers only from the retrieved article text and declines when the answer is absent [5], [12], which addresses hallucination [7]. A final check keeps the answer in Sinhala script and at a length suited to listening. Retrieval quality is examined in RQ1, faithfulness and abstention in RQ2, correctness by intent in RQ3 and language quality and format in RQ4.

3 RESEARCH METHODOLOGY

3.1 Research Design

3.1.1 Research approach and rationale

This study followed a **design-science, experimental approach**. A new artefact, a RAG pipeline, was built and then tested through experiments. Quantitative measures (retrieval accuracy, correctness, faithfulness, abstention, length compliance and language quality) were used to evaluate it. This design was used because a personalized system must be technically correct and useful to the person who listens to it.

The approach was chosen for the following reasons.

1. **Sinhala is a low-resource language.** Prompt-based control with a strong multilingual language model, supported by retrieval, is more reliable than training a model from scratch.
2. **The design fits the system architecture.** The personalization stage already produces a structured JSON record for every request. The RAG component uses this record directly as its control signal**.**
3. **The target users cannot check the printed page.** A visually impaired listener cannot verify an answer against the text. The design therefore restricts answers to the article being read and aims to reduce errors from unrelated news stories.

3.1.2 Research questions

RQ1. Retrieval. How accurately does the retriever identify the correct source article and the passage containing the key fact for Sinhala newspaper questions?

RQ2. Faithfulness and abstention. How faithfully are the generated answers to the retrieved newspaper content, and how reliably does the system decline questions that the knowledge base cannot answer?

RQ3. How does answer correctness vary across the requested intent types (direct answer, summarization, simplification, elaboration)?

RQ4. Language quality and format. To what extent do the generated answers remain in pure Sinhala, stay within the word limits, and read fluently?

3.1.3 Study setting and conceptual framework

Study setting. The study was carried out in a controlled, offline experimental setting on a development machine. The domain is Sinhala newspaper text. The intended end users are visually impaired readers who listen to articles through the voice module.

Position in the overall system. The full system has four components. Components 1 and 2 (OCR and text correction) produce the corrected text, and Component 4 converts the final answer to speech. Component 3, which is the subject of this thesis, sits between them. The input and output records exchanged with the other components are defined in Section 4.4.3.

Conceptual framework. The framework has three layers that work together.

1. **Control layer.** A *resolver function* combines the incoming signals into final settings. This is needed because signals can conflict. For example, one sample record has `style_class: Detailed` but `language_style: simple`. The precedence is: for retrieval depth, the `detail_level` flag first, then the intent default. For wording style, `correction_applied` first, then `style_class`.

2. **Retrieval layer.** The scope is decided first. Intents such as SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE and IDENTIFY_CONTENT use only the article being read. Question answering searches the current article first and uses the whole corpus only if the question is clearly outside it. Paragraphs are then selected by depth mode (see Section 5.2.3) and ranked by hybrid search.

3. **Generation layer.** A fixed prompt template combines system rules, the style instruction, an intent instruction and the ordered context. The language model generates the answer, and a Sinhala-purity check with one retry is applied.

Hybrid retrieval method. Dense retrieval finds paragraphs with similar meaning, and BM25 rewards exact matches such as names, places and numbers. Because their scores are on different scales, they are merged using Reciprocal Rank Fusion (RRF), which uses only rank positions:

```
RRF(d) = 1 / (60 + rank_dense(d)) + 1 / (60 + rank_bm25(d))
```

3.2 Data Participants or Study Materials

3.2.1 Target population and dataset description

Corpus. The corpus consisted of **500 Sinhala newspaper articles**. The articles were collected by web scraping. Each article had a headline, body text, section category and publication date. The articles are the knowledge base from which all answers are generated. Raw articles contained noise, for example invisible soft hyphens (U+00AD) inside words, stray page numbers and advertisement lines, which were removed during preparation (Section 3.3.2).

Interaction data. The nine records in the sample personalization outputs file, supplied by Component 4 (the voice module), served as the initial test cases for the input contract. They covered the intents SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE, the style classes Simple, Detailed and Step-by-step, and the detail levels brief, structured and full. These samples were extended with additional synthetic requests (Section 3.2.3).

3.2.2 Sampling and eligibility criteria

Articles. Articles were selected from Sinhala newspapers with a mix of sections (for example politics, crime, economy, sports and local news) so that different vocabulary and structure were represented.

Inclusion criteria: written in Sinhala; has a headline and body of at least two paragraphs; has a readable publication date and category; text is complete after OCR or text extraction.

Exclusion criteria: empty or very short articles; duplicates; articles with mostly advertisement or listing content; articles with heavy OCR corruption that remains after cleaning.

Evaluation questions. Questions were sampled across articles and across the question types: factual (who, where, when), explanatory (why, how) and summary-type requests.

3.2.3 Instruments, datasets and study materials

Table 3.1: Study materials by component stage

Stage	Instrument or material	Purpose
Input contract	Personalization JSON (`intent`, `style_class`, `prompt_modifier`, `personalization_flags`, `retrieved_chunk_id`, `correction_applied`), `corrected_text`, `user_id`, current `article_id`	Control signals for the RAG pipeline
Cleaning	Python scripts with Unicode normalization (NFC)	Remove soft hyphens, zero-width noise and junk lines while keeping ZWJ (U+200D) for Sinhala conjuncts
Chunking	Paragraph-based chunker	Chunk 0 is the headline and chunks 1 to n are paragraphs. IDs follow the pattern `{article_id}::p{index:03d}`. Metadata: `article_id`, `para_index`, `is_headline`, `is_lead`, `total_paras`, `section_category`, `date`
Dense index	ChromaDB with the intfloat/multilingual-e5-small embedding model,	Semantic retrieval. Indexing uses `upsert` with the chunk ID, so it

Stage	Instrument or material	Purpose
	a multilingual model that supports Sinhala (passage: and query: prefixes are applied, as E5 models expect)	can be repeated safely
Sparse index	BM25 (for example the `rank_bm25` library), saved to disk, with optional 3 to 4 character n-grams	Exact-match retrieval and handling of inflected Sinhala words
Fusion	Reciprocal Rank Fusion, k = 60, top 20 from each retriever	Merge the two rankings
Depth control	Depth modes defined in Section 5.2.3	Choose how much of the article to retrieve
Generator	gpt-5.4-mini, temperature 0 to 0.3, output length set by detail level	Produce the answer from the retrieved context only
Quality gate	Sinhala-purity check with one retry	Keep output in Sinhala
Evaluation tools	Hit@k, MRR, length-compliance script, perplexity, faithfulness check	Measure results

Output contract. The component returns `answer_text`, `article_id`, `chunk_ids_used`, `style_applied` and `detail_level_applied` to the voice module.

Prompt template. The prompt has four parts: (1) system rules (answer only from the supplied article, say so if the answer is not present, write in Sinhala); (2) the `prompt_modifier`, inserted unchanged; (3) a short instruction for the intent; (4) the ordered context with headline and date. Because the answer will be spoken, the prompt also forbids markdown, bullet symbols and emojis, and asks for numbers that are easy to say aloud. For StepByStep answers, spoken ordinal words are used instead of digits.

3.3 Experimental or Study Procedure

3.3.1 Data-collection and experimental process

The work was carried out in the order below. Each step depended on the one before it.

1. **Define the contract and resolver.** The input and output schemas and the precedence rules were fixed first.
2. **Prepare the corpus.** Articles were cleaned and filtered (Section 3.3.2).
3. **Chunk the articles** by paragraph with metadata.
4. **Build the two indexes** (dense and BM25).
5. **Implement scope selection, depth control and RRF fusion** as one `retrieve(request)` function.
6. **Implement prompt construction and generation** as a `generate(request, chunks)` function.
7. **Return the output** to Component 4.

3.3.2 Data management and quality control

Data cleaning. The cleaning pipeline removed U+00AD, zero-width characters and extra whitespace. The zero-width joiner (U+200D) was kept because Sinhala conjuncts such as the one in the word *Sri* depend on it. NFC normalization was then applied. Junk lines such as page numbers and advertisements were removed, and empty or very short articles were dropped. Headline, section category and publication date were stored as separate fields.

Chunk quality. Very long paragraphs were split at sentence ends, and very short paragraphs (under about 15 words) were merged with a neighbor. Chunk IDs were unique and followed a fixed pattern, so every answer could be traced to its source paragraph.

Identifier mapping. The sample files used placeholder IDs such as `chunk_article_1`. A small adapter mapped these to real article IDs, and an unknown ID was reported as an error and not silently guessed.

Scope control. For intents tied to the current article, retrieval was filtered by article_id. This prevented facts from other news stories from entering the answer, which is the most harmful error for a listener who cannot see the page.

Reproducibility. A fixed random seed, a low generation temperature, saved BM25 and Chroma indexes, and logged configuration (model name, embedding model, k values) were used. Idempotent indexing meant that the pipeline could be rebuilt with the same result.

Evaluation data quality. Language-model-generated questions were checked by hand for correctness and answerability before use, and the same question set was used for every run. Faithfulness checks and LLM-as-judge scoring were used only for offline evaluation and are not part of the runtime pipeline. A sample of judged answers was also checked by hand to confirm that the judge was reliable.

Limitations of the method. The corpus was limited to 500 articles, the evaluation relied on automated and LLM-based measures without a listener study, and the test set was small. These limits are stated in Sections 6.3 and 7.5, and conclusions were drawn with them in mind.

3.4 Measures and Evaluation Metrics

The test set contained labelled Sinhala newspaper questions of two kinds. *Answerable* questions had a gold source article, a reference answer and one or more key facts that a correct answer must contain. *Unanswerable* questions had no supporting article and were expected to be refused. They were divided into two types:

- **Far:** off-domain questions (for example, product specifications or general knowledge).
- **Near:** questions about a topic that appears in the knowledge base, but whose specific fact is absent from the article.

Answerable questions covered the four intent types: direct answer, summarization, simplification and elaboration. Each question was paired with the style class and

detail level that the personalization stage would normally supply. The test queries were passed to the pipeline as input. Retrieval was run without a pre-selected chunk, so that the retriever was tested on its own and did not rely on an answer given in advance.

Retrieval metrics (RQ1)

For each answerable question the ranked list of retrieved article chunks was compared with the gold annotation. Two ranks were recorded: the rank of the first chunk from the gold article (article rank), and the rank of the first gold-article chunk that contains a key fact (evidence rank). Key-fact matching was done after Unicode normalization and removal of zero-width joiners, so that spelling variants of the same Sinhala string did not cause false misses.

- **Hit@k** is the proportion of questions whose gold article (or evidence chunk) appears in the top k results, for k = 1, 3 and 5.
- **Mean Reciprocal Rank (MRR)** is the mean of 1/rank over all answerable questions, with 0 when the gold item is not retrieved.

Faithfulness and abstention metrics (RQ2)

- **Faithfulness was scored** by an LLM judge (temperature 0) as 1 when every factual claim in the answer is supported by the retrieved context, and 0 otherwise. It was computed only for answerable questions that were answered, because a refusal contains no claims to check.
- **Correct abstention rate** is the proportion of unanswerable questions for which the system returns the fixed refusal sentence. It was reported overall and separately for far and near questions, since near questions are the harder test.
- **False refusal rate** is the proportion of answerable questions that the system wrongly refuses.

Correctness by intent (RQ3)

- **Correctness was scored** by the same LLM judge against the reference answer on a 3-point scale: 2 (key facts fully agree), 1 (partly correct or

incomplete), 0 (wrong, off-topic or missing the key fact). A false refusal was scored 0. The mean score was reported per intent type.

- **Key-fact rate** is a rule-based check, the proportion of answerable questions whose answer contains the annotated key fact. It does not depend on the judge and was used to cross-check it.

Language quality and format metrics (RQ4)

- **Sinhala purity** is the fraction of the answer made up of Sinhala script, from the same check used inside the generation layer. The mean score and the proportion of answers at or above 0.85 were reported.
- **Word-limit compliance** is the proportion of answers whose length is within the maximum set for the request.
- **Perplexity was used as** an indicator of fluency. It was computed only for answered questions, because the refusal sentence is fixed and would give an artificially low value. It was reported overall, per intent, and separately for correct and incorrect answers. Perplexity measures how predictable the text is. The perplexity score was produced by the OpenAI model gpt-5.4-mini, using the token log-probabilities that it returns. It does not measure factual correctness, and it was interpreted alongside the other metrics.

3.5 Data Analysis

3.5.1 Data sources and analysis plan

Three data sources were analysed: the cleaned corpus of 500 articles, the labelled test set of 72 queries (56 answerable and 16 unanswerable, of which 8 are far and 8 are near), and the run logs produced by the pipeline for each query. Table 3.2 links each research question to its data and analysis.

Table 3.2: Mapping of research questions to analysis

RQ	Data analysed	Measures

RQ1	Ranked chunk IDs for 56 answerable queries; gold article and evidence annotations	Hit@1, 3, 5; MRR (article and evidence level)
RQ2	Answered or refused flag for all 72 queries; judge verdicts	Faithfulness, correct abstention (overall, far, near), false refusal rate
RQ3	Judge scores and key-fact flags for answerable queries	Mean correctness and key-fact rate per intent
RQ4	Generated answers for answered queries	Sinhala purity, word-limit compliance, perplexity

3.5.2 Corpus profiling

The cleaned corpus was described before the evaluation so that readers can judge how far the results may generalise. The following were reported: the number of articles per newspaper and per section category, the publication date range, the number of paragraphs per article, the total number of chunks, and the chunk length in words (mean, median, minimum and maximum). The counts before and after cleaning and filtering were also reported.

3.5.3 Consolidation of evaluation data

Every query was stored as one row in a single results table, which was the only input to the analysis scripts. Each row contained the query ID, query type (answerable, far or near), intent, style class, detail level, gold article ID, gold evidence chunk, the retrieved chunk IDs in rank order, article rank, evidence rank, refusal flag, generated answer, faithfulness verdict, correctness score, key-fact flag, Sinhala purity, word count, word limit and perplexity.

The following rules were applied before any metric was computed:

- Text was normalised (NFC) and zero-width joiners were removed before key-fact string matching.
- A refusal was identified by an exact match with the fixed refusal sentence.

- If the gold item was not retrieved, its reciprocal rank was set to 0.
- Faithfulness and perplexity were computed only for answered answerable queries.

The table was saved with the raw generated answers, so the whole analysis could be repeated on identical outputs.

3.5.4 Retrieval analysis (RQ1)

Hit@k (k = 1, 3, 5) was calculated as the share of the 56 answerable queries whose gold item appeared within the top k results. MRR was calculated as:

$$MRR = (1 / |Q|) \times \Sigma (1 / rank_i)$$

Both were computed at two levels, the source article and the evidence passage. The difference between the two levels was analysed as a diagnostic. A large gap at rank 1 indicates that the retriever finds the correct article but ranks a neighboring passage above the one containing the answer.

3.5.5 Faithfulness and abstention analysis (RQ2)

The 72 queries were cross-tabulated by query type (answerable, far, near) and system behavior (answered, refused). From this table the following were derived, each with its denominator stated:

- the false refusal rate (over the 56 answerable queries);
- the correct abstention rate (over the 16 unanswerable queries), reported overall and for far and near queries separately;
- faithfulness (over the answerable queries that were answered).

To examine the cause of false refusals, each falsely refused query can be classified by whether its evidence passage was retrieved in the top five, which shows whether refusals follow retrieval misses or arise at the generation stage. This classification was not carried out in the present evaluation (Section 7.1) and is listed as future work (Section 9.4).

3.5.6 Correctness analysis by intent type (RQ3)

The mean judge score (0 to 2) was calculated for each intent type, together with the number of queries in each group and the distribution of scores 0, 1 and 2. The rule-based key-fact rate was calculated for the same groups. Comparing the judge-based and rule-based measures shows whether the judge is stricter or more lenient than exact matching. False refusals were scored 0 and were therefore included in the means.

3.5.7 Language quality and format analysis (RQ4)

For answered queries, the mean Sinhala purity and the proportion of answers with purity of at least 0.85 were reported, along with the proportion of answers within the word limit. Perplexity was reported as mean and median overall, per intent, and separately for correct and incorrect answers. This last comparison tested whether fluency is related to correctness. Perplexity was interpreted only as an indicator of fluency and not of factual accuracy.

3.6 Ethics Privacy Security and Responsible AI

3.6.1 Ethical approval and informed consent

Use of published articles. The newspaper articles are copyrighted. They were used only for non-commercial academic research, were cited by source and date, and were not republished in full in the thesis or in any shared material.

3.6.2 Privacy, security and data governance

Data classification and handling.

Table 3.3: Data classification and handling

Data	Sensitivity	Handling
Newspaper articles and chunks	Public, copyrighted	Stored locally, used for research only, not redistributed in full
Test requests, reference answers, generated answers	Low	Stored with the code and results

Use of an external language model. Answers were generated by gpt-5.4-mini through an online service. This meant that article text and the request text left the local machine. To reduce the risk, only public article text and the style and request text were sent. User IDs were never included in a prompt. The provider's data-use and retention terms apply to this processing and would need to be reviewed before any deployment (Section 8.4.3).

Security measures.

- API keys are kept in environment variables and never saved in the code or in the shared repository.
- Data files are on an encrypted or password-protected device and are backed up securely.
- Access to the raw data is limited to the researcher and the supervisor.
- Run logs do not contain keys or personal data.

3.7 Reproducibility and Research Artefacts

3.7.1 Data, code and materials availability

Table 3.4: Availability of research artefacts

Artefact	Availability
Source code (cleaning, chunking, indexing, retrieval, generation, evaluation)	Kept in a version-controlled repository at *https://github.com/Janindu-Muthunayaka/R26-DS-002*, shared with the examiners and made public after the assessment if permitted
Configuration files and prompt templates	Included in the repository
Personalization sample JSON and the extended test requests	Included
Reference answers and evaluation scripts	Included
Raw newspaper articles	**Not shared in full** because of copyright. The repository lists article identifiers, sources and dates, and includes the cleaning script, so that others can rebuild the corpus from the

Artefact	Availability
	same sources
Built indexes (Chroma and BM25)	Rebuilt from the cleaned corpus by one script. They are not shared if they contain full copyrighted text

3.7.2 Documentation, versioning and computational environment

Documentation. The repository has a README that explains the folder layout, the order of the scripts, and how to reproduce each experiment. The input and output schemas, the precedence rules in the resolver, and the prompt templates are documented, and each experiment has its own script or notebook.

Versioning. All code and configuration are stored in Git, with a tag for the version used to produce the thesis results. Cleaned data and prompt files are versioned, and a SHA-256 hash of the cleaned corpus is recorded so that it is clear which data produced which result.

Configuration. All settings are in one configuration file so that none are hidden in the code: embedding model, scoring model, chunking limits, retrieval parameters (top 20 per retriever, RRF constant 60), depth modes, generation temperature and maximum length, the random seed, and the style and length rules.

Computational environment. The following are recorded in the thesis: the operating system, the Python version, the hardware (CPU, GPU if any, memory), and the versions of the main libraries (ChromaDB, rank_bm25, Hugging Face Transformers, PyTorch, pandas, SciPy and statsmodels), which are fixed in a requirements file. The name and version of the generation model, and the dates of the generation runs, are also recorded.

Randomness. Random seeds are fixed for sampling, bootstrapping and simulation. The generation temperature is low (0 to 0.3), and the raw generated answers are saved so that the analysis can be repeated on exactly the same outputs.

Run logging. Each run saves its configuration, retrieved chunk IDs, prompts, outputs and scores in the results table described in Section 3.5.

3.7.3 Limitations on reproducibility

4. **Hosted language model.** The generation model is accessed through a service that can update or withdraw models. Even at a low temperature, the output is not guaranteed to be identical on every run. To reduce this problem, the version and date are recorded, and the generated answers are saved and shared.
5. **Copyrighted corpus.** The full articles cannot be redistributed, so another researcher must rebuild the corpus from the listed sources, and small differences in the collected text are possible.
6. **Library and hardware differences.** Different versions of the embedding model, vector database or hardware can change the numeric values slightly, even when the code is the same.

These limits do not affect the comparison between conditions in this study, because all conditions share the same data, models and settings. They do affect how exactly another researcher can repeat the absolute numbers, and the thesis states this clearly.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Stakeholders and user needs

The system is designed for people who cannot read printed newspapers and who listen to the content instead. Because the output is spoken, a wrong or confusing answer cannot be checked visually, so accuracy and clarity matter more than in a normal text interface.

Table 4.1: Stakeholders and their main needs

Stakeholder	Role	Main needs
Visually impaired newspaper reader (primary user)	Listens to articles and asks spoken questions	Short answers when in a hurry, full details when interested, simple wording when the text is difficult, step-by-step explanation when needed, and no mixing of facts from different news stories
Caregivers and family members	Help set up and sometimes use the system	Easy setup, predictable behavior, and trustworthy answers
Organizations for the blind and libraries	Possible deployers	Low running cost, Sinhala-language support, and no need for specialized hardware

The main user needs identified from the project scenario and from the sample personalization outputs are:

1. **Control over answer length.** Users sometimes want only the main points and sometimes the complete story (the *brief*, *structured* and *full* detail levels).
2. **Control over wording.** Users want simple words, a detailed explanation, or numbered steps (the *Simple*, *Detailed* and *StepByStep* style classes).
3. **Faithfulness to the article.** Answers must come from the article being read and not from the model's memory or from other articles.

4. **Audio-friendly text.** The answer must be easy to listen to, without symbols or formatting that a speech engine would read badly.

4.1.2 Functional requirements

Table 4.2: Functional requirements

ID	Requirement
FR1	The component shall accept the corrected Sinhala text from Component 2 together with the personalization record from Component 4 (route, intent, style class, prompt modifier, personalization flags, retrieved chunk identifier, and applied correction), the user identifier, and the identifier of the article being read.
FR2	The component shall clean and normalise the newspaper corpus, including removal of invisible characters (such as the soft hyphen) while preserving characters needed for Sinhala conjuncts.
FR3	The component shall divide each article into paragraph-level chunks with metadata (article identifier, paragraph index, headline and lead flags, section, and date).
FR4	The component shall maintain a dense (semantic) index and a keyword (BM25) index over the same chunks.
FR5	The component shall retrieve passages with hybrid search, running both retrievers for each query and combining their rankings with Reciprocal Rank Fusion (RRF).
FR6	The component shall limit retrieval to the current article for summarising, explaining, simplifying, elaborating and identifying intents, and shall search the whole corpus only when the question is outside the current article or no article is active.

ID	Requirement
FR7	The component shall always pass the retrieved paragraphs to the generator in their original article order.
FR8	The component shall inject the prompt modifier into the generation prompt and apply the intent-specific instruction.
FR9	The component shall resolve conflicting personalization signals using a fixed precedence rule (for example, a style class of *Detailed* combined with a language style of *simple*).
FR10	The component shall produce speech-ready text and shall verify that the answer is in Sinhala, retrying once if it is not.

4.1.3 Non-functional requirements

Table 4.3: Non-functional requirements

ID	Category	Requirement
NFR1	Faithfulness	Answers shall be grounded in the retrieved text. Unsupported statements shall be minimised and measured during evaluation.
NFR2	Performance	The complete answer should be produced within a few seconds for a typical article, because the user is waiting by voice.
NFR3	Language quality	Output shall be natural, grammatical Sinhala, with a purity check to detect responses in the wrong language.
NFR4	Accessibility	Output shall be suitable for text-to-speech, with short sentences, spoken forms of numbers, and numbered steps written as

ID	Category	Requirement
		spoken words.
NFR5	Modularity	The component shall communicate with other components only through a defined JSON contract, so that each component can be tested and replaced separately.
NFR6	Reproducibility	Indexing shall be idempotent (the same chunk identifier always replaces the same record), and the retrieved chunk identifiers shall be returned for debugging and evaluation.
NFR7	Cost and resources	The design shall not need GPU training. It shall run on ordinary hardware, with a hosted language model for generation.
NFR8	Scalability	The design shall work for the research corpus of about 500 articles and shall be able to grow by adding new chunks without rebuilding the whole system.
NFR9	Maintainability	Retrieval, prompt building and generation logic shall be separate modules.

4.1.4 Constraints and assumptions

Constraints

- **Low-resource language.** Sinhala has fewer ready-made embedding models, tokenizers and evaluation sets than English. Sinhala is also highly inflected, so one word has many surface forms (for example *සැකකරු*, *සැකකරුවන්*, *සැකකරුගේ*), which weakens exact keyword matching.

- **Noisy corpus.** The sample articles contained soft hyphens inside words, stray page numbers and advertisement text that came from the source and from OCR.
- **Fixed upstream formats.** The personalization record is produced by another component, so Component 3 must adapt to its fields (for example, identifiers such as `chunk_article_1` must be mapped to real article identifiers).

Assumptions

- Component 2 delivers corrected Sinhala text of acceptable quality.
- The personalization record from Component 4 is valid and always has `route`, `intent` and `style_class`. The `detail_level` flag is optional.
- The user is usually reading one article at a time, and the system knows which article is active.
- The hosted language model can follow Sinhala instructions and the prompt modifier reliably.

4.2 Component Architecture within the Complete Solution

4.2.1 Overall solution architecture

The complete solution is made of four components that form a pipeline from a printed newspaper page to a spoken answer.

Table 4.4: Responsibilities of the four components

Component	Responsibility
Component 1	Reads the newspaper image and produces Sinhala text using OCR.
Component 2	Corrects OCR errors and passes the corrected text onward.
Component 3	Retrieves the relevant article content and generates a personalized Sinhala answer.
Component 4	Handles voice interaction: it receives the user's spoken request, produces the personalization record (intent, style

Component	Responsibility
	class, prompt modifier, detail level, and so on), and reads the final answer aloud.

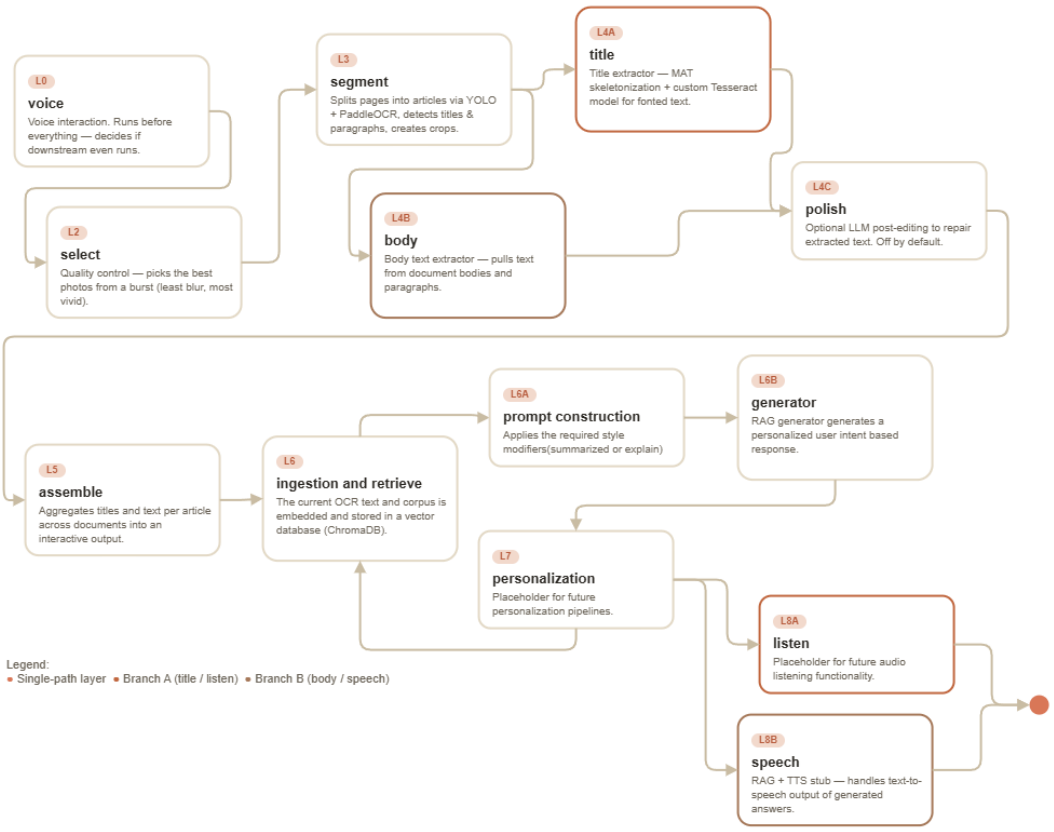


Figure 4.1: Overall solution architecture

4.2.2 Internal architecture of Component 3

Inside Component 3, the design has two phases:

- **indexing phase.** The article corpus is cleaned, divided into paragraph chunks, and loaded into two indexes (a dense vector index in ChromaDB and a BM25 keyword index).

- **answering phase.** For each request, the component resolves the personalization signals, decides the retrieval scope and depth, runs retrieval, builds the prompt, generates the answer, checks it, and returns the result.

Table 4.5: Modules of Component 3

Module	Responsibility	Main inputs	Main outputs
Request adapter and signal resolver	Reads the input, maps chunk identifiers to article identifiers, and resolves conflicting signals using fixed precedence rules	Corrected text, personalization record, user and article identifiers	Normalised request object
Corpus cleaner	Removes soft hyphens, zero-width characters (while keeping the zero-width joiner that Sinhala conjuncts need), junk lines and very short articles; applies Unicode NFC normalization	Raw articles	Clean articles with headline, section and date
Chunker	Creates a headline chunk and paragraph chunks with metadata; splits very long paragraphs and merges very short ones	Clean articles	Chunks with identifiers in the form `{article_id}::p{index}`
Dense index	Stores chunk embeddings with metadata for semantic search, using upsert for idempotent indexing	Chunks	Vector collection
BM25 index	Stores keyword statistics, with an optional character	Chunks	Saved keyword index

Module	Responsibility	Main inputs	Main outputs
	n-gram field to handle Sinhala inflection		
Scope selector	Chooses between the current article and the whole corpus based on intent and article context	Normalised request	Search filter
Hybrid retriever	Runs dense and BM25 search and merges the two rankings with Reciprocal Rank Fusion	Query, filter	Ranked chunk list
Prompt builder	Builds the prompt from system rules, the injected prompt modifier, the intent instruction, ordered context and voice-output rules	Request, ordered chunks	Final prompt
Generator and checker	Calls the language model at low temperature, limits length by detail level, checks Sinhala purity and retries once if needed	Prompt	Answer text
Output adapter	Packages the result for Component 4	Answer, trace data	Output JSON

4.2.3 Data flow and system interactions

The flow for one request is as follows.

1. Component 4 sends the corrected text, the personalization record, the user identifier and the active article identifier to Component 3.

2.The request adapter validates the input and maps the chunk reference to a real article. The signal resolver decides the final detail level and wording style.

3. The scope selector sets the search filter. Intents such as SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE and IDENTIFY_CONTENT stay within the current article.

4.The retrieval depth controller selects the chunks:

- *brief*: headline, lead paragraph, and one or two best paragraphs from a small hybrid search;
- *structured*: all paragraphs of the article in original order, within a token budget;
- *full*: hybrid search (using the headline as the query) with neighboring paragraphs added;
- question answering: hybrid search for the question itself.

5.For any search, dense and BM25 retrievers each return their top 20 results, which are merged with RRF using the formula RRF(d) = 1/(60 + rank_dense(d)) + 1/(60 + rank_bm25(d)). Duplicates are removed and the chunks are sorted by paragraph index.

6.The prompt builder assembles the prompt, and the generator produces the answer. The Sinhala purity check runs, with one retry if it fails.

7. The output adapter returns `answer_text`, `article_id`, `chunk_ids_used`, `style_applied` and `detail_level_applied` to Component 4, which converts the answer into speech.

Interfaces. Component 3 depends on two external services: a text embedding model and a hosted language model. Both are used through a thin wrapper, so they can be replaced without changing the rest of the design. All communication with other components uses JSON.

4.2.4 Architecture rationale

- **Separation of indexing and answering.** Cleaning, chunking and indexing are done once and offline, which keeps answering fast and makes the indexes reproducible.
- **Paragraph-level structure.** The *brief structured* and *full* modes need to know which paragraph is the lead, which comes next and which is the last. Paragraph chunks with metadata make this possible.
- **Article scoping before searching.** Restricting retrieval to the current article prevents facts from other stories from entering the answer. This is the most dangerous error for a user who cannot see the page.
- **Two retrievers combined by rank.** Newspaper text is full of personal names, places and numbers, which keyword search handles well, while dense search handles meaning and wording differences. RRF uses ranks and not raw scores, so the two scales do not need to be matched.
- **Personalization in two places.** Most RAG systems change only the prompt. This design also changes *what is retrieved* and therefore forms the main research contribution: style-aware retrieval depth.
- **Prompt-based personalization.** The prompt modifier carries style information as text, so no model training is needed. This suits a low-resource language with a limited budget.
- **Output contract.** The fixed JSON contract allows each of the four components to be developed and tested on their own.

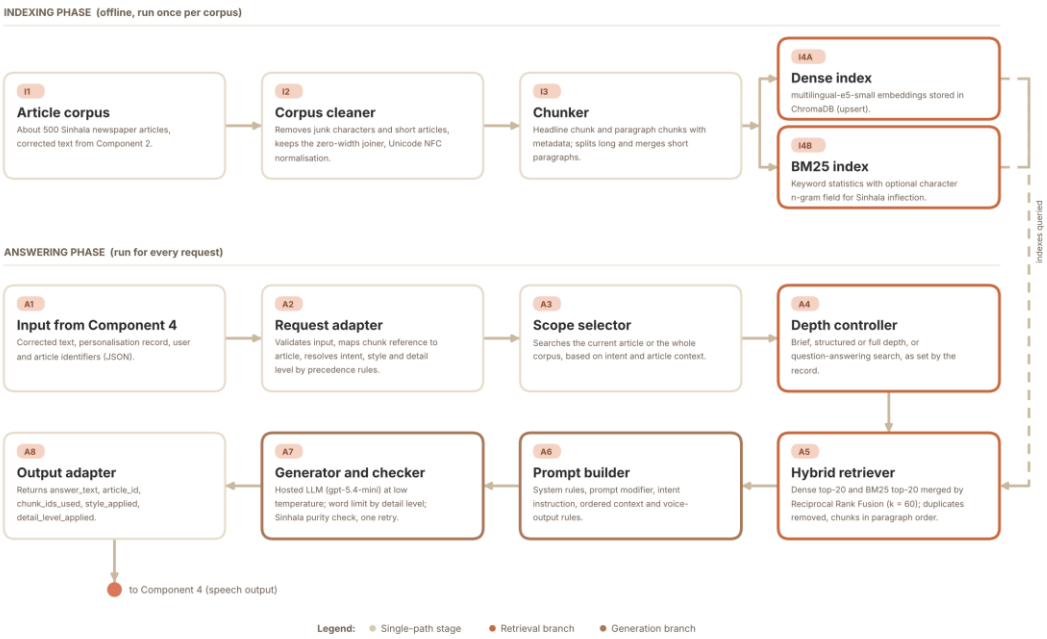


Figure 4.2: Component 3 RAG system architecture

4.3 Design Alternatives and Decisions

4.3.1 Alternatives considered

Four main design decisions were examined. For each, the alternatives are listed below.

Table 4.6: Design decisions and alternatives considered

Decision	Alternative A	Alternative B	Alternative C	Alternative D
D1: How to personalize answers	Fine-tune or LoRA-train the model for each style	Prompt modifier only	Prompt modifier + retrieval depth control	
D2: How to retrieve	Dense search only	BM25 only	Dense search with BM25 only as a fallback	Hybrid search with both retrievers combined by RRF
D3: How to divide articles	Fixed-size chunks	Sentence-level chunks	Paragraph-level chunks with metadata	
D4: How to	Local open-source	Hosted language		

Decision	Alternative A	Alternative B	Alternative C	Alternative D
generate	language model	model (gpt-5.4-mini)		

The main reason each alternative was considered is as follows.

- **D1-A (fine-tuning):** gives strong control over style but needs training data and GPUs that are not available for Sinhala.
- **D1-B (prompt only):** is simple and cheap, but changes only the wording and cannot change how much content the user hears.
- **D2-A and D2-B:** are the simplest retrieval methods. Dense search can miss exact names, and BM25 can miss inflected word forms.
- **D2-C (fallback):** Keyword search is used only when the dense result is not sufficient, so the two retrievers are not truly combined.
- **D3-A and D3-B:** are common in RAG systems, but they ignore the natural structure of a newspaper article.
- **D4-A:** gives control and privacy, but Sinhala generation quality and the infrastructure cost are concerns.

4.3.2 Selected design and justification

The selected design is a **RAG pipeline** with these main choices:

5. **Personalization** through a prompt modifier and adaptive retrieval depth (D1-C).
6. **Retrieval** through dense and BM25 search combined with Reciprocal Rank Fusion, limited to the current article where appropriate (D2-D).
7. **Chunking** by paragraph with rich metadata, so the system always knows the position of each passage (D3-C).
8. **Generation** with a hosted language model at low temperature, with voice-friendly output rules and a Sinhala purity check (D4-B).

These choices were selected because together they:

- satisfy all the *Must* requirements
- work within the constraints of a low-resource language and a limited budget;

- keep answers grounded in the article, which protects a user who cannot verify the text visually;
- provide a clear research contribution that can be tested in future work through ablation (prompt modifier only, retrieval depth only, and both together), which is outside the scope of this study (Section 1.7); and
- keep the component modular, with a fixed JSON contract to the other three components.

4.4 Detailed Design

4.4.1 User-interface and user-flow design

Interface approach. The end users are visually impaired, so the main interface is **voice only**, and it is provided by Component 4. Component 3 has no screen of its own. Its "interface" is the spoken answer that it prepares, and the JSON contract through which it receives requests and returns answers. The design therefore focuses on how the answer *sounds* and how the conversation *flows*.

Design principles for spoken answers

Table 4.7: Design principles for spoken answers

Principle	How it is applied
One idea per sentence	Short sentences, with the length controlled by the *Simple* style class.
No visual symbols	No markdown, bullets, tables, brackets or emojis, because a speech engine would read them badly or skip them.
Spoken structure	Steps are announced with spoken ordinal words (for example "පළමුව, දෙවනුව") and not with "1." or "2.".
Numbers and dates	Written in a form that is easy to speak and understand.
Predictable length	*Brief* answers are about 2 to 4 sentences. *Full* answers are longer but remain in the original article order.
Honest gaps	If the answer is not in the article, the system says so clearly and does not

Principle	How it is applied
	guess.
Same voice for the same article	The answer names the article headline when the user asks what they have (identify-content intent).

User flow. A typical session follows these steps:

1. The user scans a newspaper page (Components 1 and 2 produce the corrected text).
2. The user asks for a spoken request, such as "Summarize this" or "Walk me through this step by step".
3. Component 4 recognizes the request and prepares the personalization record. Component 3 produces the answer.
4. Component 4 reads the answer aloud.
5. The user may then give feedback or a new request, such as "Explain it more simply", which is treated as a correction.

Developer and evaluation interface. For development and testing, a simple command-line or notebook interface lets the researcher send a request JSON and view the answer, the style and detail level applied, and the chunk identifiers used. This interface is not part of the product used by the end user.

4.4.2 Use cases and process flows

Actors: the reader (primary), Component 4 (voice module), Component 2 (text correction), and the researcher (for evaluation).

Table 4.8: Use cases of Component 3

ID	Use case	Trigger (intent and flags)	Main flow	Alternative or exception
UC1	Quick summary	SUMMARIZE, `detail_level: brief`	Fetch headline and lead paragraph, add one or two best paragraphs,	If the article has only one paragraph, use it alone

ID	Use case	Trigger (intent and flags)	Main flow	Alternative or exception
			generate a short answer in a simple style	
UC2	Step-by-step explanation	EXPLAIN, style *StepByStep*, `detail_level : structured`	Fetch all paragraphs in order, generate numbered spoken steps	If the article is longer than the token budget, trim the paragraphs furthest from the lead
UC3	Full details	ELABORATE, style *Detailed*, `detail_level : full`	Hybrid search with the headline as the query, add neighboring paragraphs, generate a thorough answer	If search returns few results, fall back to all paragraphs
UC4	Simple explanation	EXPLAIN or SIMPLIFY, style *Simple*	Retrieve according to the detail level and apply the simple-words modifier	If signals conflict (for example *Detailed* with language style *simple*), apply the precedence rule
UC5	Identify content	IDENTIFY_CONTENT	Use the headline, section and lead paragraph to tell the user what they are holding	If the headline is missing, use the lead paragraph only
UC6	Question about the article	QUESTION_ANSWER	Hybrid search for the question within the current article	If nothing relevant is found, say that the article does not contain the answer

ID	Use case	Trigger (intent and flags)	Main flow	Alternative or exception
UC7	Question outside the article	QUESTION_ANSWER, no match in the current article, or no active article	Hybrid search across the whole corpus, and name the source article in the answer	If still nothing is found, say so clearly
UC8	Style correction	`correction_applied` set	Apply the correction immediately	

Main process flow (answering a request)

1. Receive and validate the request.
2. Map the chunk reference to an article identifier.
3. Resolve the detail level and wording style using the precedence rules.
4. Choose the scope (current article or whole corpus).
5. Retrieve chunks according to the detail level (fetch by order, or hybrid search with fusion).
6. Add neighboring paragraphs where required, remove duplicates, and sort by paragraph index.
7. Build the prompt, then generate the answer.
8. Run the Sinhala purity and length checks; retry once if a check fails.
9. Return the output JSON.

4.4.3 Database and data-model design

The component uses two stores, each chosen for the kind of data it holds: a vector collection and a keyword index for retrieval.

Article record (after cleaning)

Table 4.9: Article record after cleaning

Field	Type	Description
`article_id`	string	Unique identifier of the article

Field	Type	Description
`headline`	string	Cleaned headline
`section_category`	string	Newspaper section (for example politics or sports)
`publication_date`	date	Date of the issue
`paragraphs`	list of strings	Cleaned body paragraphs in original order

Chunk record (stored in the dense index and the BM25 index)

Table 4.10: Chunk record

Field	Type	Description
`chunk_id`	string	Format `{article_id}::p{index:03d}`; also the vector-store identifier, so re-indexing replaces the same record
`text`	string	Cleaned paragraph text
`article_id`	string	Parent article
`para_index`	integer	Position in the article (0 is the headline)
`is_headline`	boolean	True for chunk 0
`is_lead`	boolean	True for the first body paragraph
`total_paras`	integer	Number of paragraphs in the article
`section_category`	string	Copied from the article
`date`	date	Copied from the article
`embedding`	vector	Stored only in the dense index; built with the passage: prefix, as the E5 embedding model expects

Input record (from Components 2 and 4)

Table 4.11: Input record

Field	Type	Notes

Field	Type	Notes
`corrected_text`	string	The user's request, corrected by Component 2
`route`	string	For example `GENERATE`
`intent`	string	SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE, IDENTIFY_CONTENT or QUESTION_ANSWER
`style_class`	string	*Simple*, *Detailed* or *StepByStep*
`prompt_modifier`	string	Style instruction injected into the prompt
`personalization_flags`	object	May contain `detail_level` (brief, structured or full) and `language_style`
`retrieved_chunk_id`	string	Upstream chunk reference, mapped by the adapter
`correction_applied`	string or null	The style the user asked for as a correction
`user_id`	string	Identifies the user
`article_id`	string	The article being read

Output record (to Component 4)

Table 4.12: Output record

Field	Type	Notes
`answer_text`	string	Speech-ready Sinhala text
`article_id`	string	Article used
`chunk_ids_used`	list of strings	Trace of the chunks given to the model
`style_applied`	string	Final wording style after conflict resolution
`detail_level_applied`	string	Final detail level after conflict resolution

Exception and error handling

The guiding rule is that a visually impaired user must never hear silence or a confusing answer. Every failure therefore leads either to a safe fallback or to a short, clear spoken message that Component 4 can read.

Table 4.13: Exception and error handling

Situation	Detection	Handling
Missing or invalid fields	Request validation	Return an error record with a code; use defaults for optional fields (for example no `detail_level`)
Unknown chunk alias or article	Lookup fails in `chunk_alias` or the article store	Use the active `article_id` if provided; otherwise search the whole corpus; if still not found, return a spoken "article not found" message
Conflicting signals	Resolver	Apply the precedence rules and record the final values in the output
Empty or poor retrieval	No results or very low fusion scores	Widen the scope from the current article to the whole corpus; if still empty, say that the information is not available
Article too long for the model	Token count above budget	Trim the paragraphs furthest from the lead and keep the headline and lead
Language-model timeout or service error	Timeout or error response	Retry once; then return the lead paragraph as a plain fallback with a spoken notice
Answer not in Sinhala, or wrong length	Purity and length checks	Retry once with a stricter instruction; if it fails again, return the best available answer or the fallback
Unexpected internal error	Top-level exception handler	Log the error code without personal content

Situation	Detection	Handling
		and return a generic spoken error message

4.5 Standards and Professional Practice

Component 3 supports visually impaired users, uses a hosted language model and works with published newspaper text. For this reason, accessibility, privacy, copyright and trustworthy AI were treated as design concerns, and the relevant standards were used as guidance. Formal certification is not claimed.

Standards and regulations. The design follows ISO/IEC/IEEE 29148 for requirements, ISO/IEC 25010 for quality characteristics, ISO/IEC/IEEE 12207 for the development life cycle, and the 29119 series for test planning. Sinhala text handling follows the Unicode Standard (NFC normalization, UTF-8, preservation of the zero-width joiner) and ISO 639-1 language tagging. WCAG 2.2 and the UN Convention on the Rights of Persons with Disabilities Guide Accessibility. Sri Lanka's Personal Data Protection Act (2022) and Intellectual Property Act (2003) guide privacy and copyright. OWASP's Top 10 for LLM applications guides security, while ISO/IEC 23894, the NIST AI RMF and ISO/IEC 42001 guide AI risk management. The ACM and IEEE codes of ethics guide professional conduct.

Security, privacy, accessibility and ethics. Input is validated, article text is treated as data and not as instructions, and secrets are kept out of code and logs. Spoken requests and answers are not logged. Answers are written for listening, users control the level of detail, and the system states clearly when information is missing. The main ethical risks are unfaithful answers, over-trust, over-simplification, bias, unequal user ability and copyrighted text. Each has a planned response: grounding in retrieved text, scoping to the current article, low temperature and research-only use of the corpus.

Development process and quality assurance. The component was built iteratively through nine stages, from the data contract to evaluation, and each stage was reviewed with the supervisor. Practices include Git version control, separation of configuration from code, reproducibility (idempotent indexing, recorded model versions, a chunk trace in every output), modular documented Python, and maintained documentation.

Limitations and compliance. The system was evaluated on about 500 articles, depends on upstream OCR and correction quality, and relies on a hosted model. Its evaluation used LLM-judged measures and automated metrics without a listener study, so the results indicate trends and not statistical certainty. It is intended for news reading only, not for medical, legal or financial advice. The researcher states that practice followed recognized engineering and ethics principles, that the corpus is used only for non-commercial research, and that results are reported honestly.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

5.1.1 Development environment

Development and local execution were performed on a Lenovo 82KB laptop running 64-bit Windows 11 Home Single Language. The system has an 11th Gen Intel Core i5-1135G7 processor (4 cores, 8 threads, 2.4 GHz) and 8 GB of RAM. Graphics are provided by integrated Intel Iris Xe; no dedicated GPU was reported.

The application runs locally with Python 3.13.0. ChromaDB stores the vector index locally, and the `intfloat/multilingual-e5-small` embedding model is used for embeddings. Answer generation uses OpenAI's `gpt-5.4-mini` API, so generation requires internet access and runs on the hosted service rather than being performed by the laptop's GPU. These are the observed development-machine specifications, not minimum deployment requirements.

5.1.2 Technology stack

Table 5.1: Technology stack

Layer	Technology	Purpose
Language	Python 3.13.0	All pipeline code
Text processing	`unicodedata`, `re`	Sinhala cleaning, NFC normalization
Embedding model	intfloat/multilingual-e5-small	Dense vectors for Sinhala chunks and queries
Vector database	ChromaDB	Dense index with metadata filtering
Keyword index	`rank_bm25`	Exact-match retrieval of names, places, numbers
Generator	gpt-5.4-mini (API)	Answer generation in Sinhala
Version control	Git + GitHub	Source-code management

5.1.3 Dependencies, licenses and configuration

All libraries are open source and used under their own licenses: ChromaDB (Apache 2.0), rank_bm25 (Apache 2.0), sentence-transformers (Apache 2.0). The embedding model is used under its published license (MIT). The language model is accessed as a

paid API service under the provider's terms of use, and the 500 newspaper articles are used only for academic research and are not redistributed.

Dependencies are pinned in `requirements.txt`. Configuration is separated from code in a `config.yaml` file and environment variables: chunk-size limits, top-k values, the RRF constant (k = 60), generation temperature, and output length per detail level. The API key is read from an environment variable and is never written in the source code.

5.1.4 Development process and source-code management

An iterative, incremental process was followed. Each increment produced a testable unit before the next one started: (1) input/output contract, (2) corpus cleaning, (3) paragraph chunking, (4) dense and BM25 indexes, (5) retrieval scope and depth control, (6) RRF hybrid search, (7) prompt building and generation, (8) output adapter, (9) evaluation. This order was chosen because each step depends on the output of the previous one, and the data pipeline errors (for example invisible characters in Sinhala text) would otherwise spread into every later stage.

Git was used with a `main` branch for stable code and feature branches such as `feature/chunking`. Commits were small and had descriptive messages. The agreed JSON contract with Components 2 and 4 was kept in one shared schema file, so changes to it were tracked in version control.

5.2 Component Implementation

5.2.1 Implementation overview

Component 3 receives the corrected text from Component 2 and a personalization JSON from Component 4. The JSON contains `intent`, `style_class`, `prompt_modifier`, `personalization_flags`, `retrieved_chunk_id` and `correction_applied`. It also receives the `user_id` and the article the user is currently reading. The component returns `answer_text`, `article_id`, `chunk_ids_used`, `style_applied` and `detail_level_applied` to the voice module.

The processing flow is:

1. **Request resolver:** merges conflicting signals into one final style and detail level.
2. **Scope selection:** decides whether to search the current article or the whole corpus.
3. **Adaptive retrieval depth control:** chooses which paragraphs to fetch, based on `detail_level`.
4. **Hybrid retrieval:** dense search and BM25 fused by RRF when search is needed.
5. **Prompt construction:** injects the style modifier and intent instruction.
6. **Generation and checks:** produces a Sinhala answer suitable for speech.

The main contribution of research is that personalization changes **what is retrieved** (retrieval depth) and not only **how the answer is worded** (prompt).

5.2.2 Data pipeline and storage implementation

Corpus cleaning. The 500 articles contained invisible soft hyphens (U+00AD) inside Sinhala words, stray page numbers and advertisement text. These break keyword matching and weaken embeddings. The cleaning function removes U+00AD, zero-width spaces and extra whitespace, but keeps the zero-width joiner (U+200D) because Sinhala conjunct letters such as ක්‍ depend on it. Text is then normalised to Unicode NFC. Junk lines are removed and empty or very short articles are skipped. Headline, section category and publication date are stored as separate fields.

Paragraph chunking. Fixed-size chunks were rejected because newspaper meaning follows paragraph structure and the depth control needs paragraph order. Chunk 0 is the headline and chunks 1 to n are body paragraphs. The ID format is `{article_id}::p{index:03d}`. Each chunk stores `article_id`, `para_index`, `is_headline`, `is_lead`, `total_paras`, `section_category` and `date`. Very long paragraphs are split at sentence ends, and paragraphs under about 15 words are merged with a neighbor.

Dense index. Each chunk is embedded and stored in ChromaDB with its metadata. The chunk ID is used as the Chroma ID and `upsert` is used, so re-running indexing never creates duplicates. For E5-type models, the `passage:` prefix is added at indexing time and the `query:` prefix at query time, as these models expect.

BM25 index. ChromaDB has no native BM25, so a separate `rank_bm25` index is built from the cleaned text, saved to disk, and loaded at startup. Tokens are made by whitespace splitting and punctuation stripping. Because Sinhala is highly inflected (for example සැකකරු, සැකකරුවන්, සැකකරුගේ), character 3–4-grams are added as an extra field so that inflected forms of the same word still match.

5.2.3 Algorithm and business-logic implementation

A. Request resolver. The samples contain conflicts, for example `style_class: Detailed` together with `language_style: simple`. One function applies a fixed precedence:

Table 5.2: Priority rules of the request resolver

Decision	Priority (highest first)
Retrieval depth	`detail_level` flag, then intent default
Wording style	`correction_applied`, then `style_class`

B. Retrieval scope. For SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE and IDENTIFY_CONTENT, only the current article is searched (metadata filter on `article_id`). For question answering, the current article is searched first, and the whole corpus is used only if the question is clearly outside it or no article is active. This prevents facts from other news stories being mixed in, which is the most dangerous error for a user who cannot see the page. A small adapter maps `retrieved_chunk_id` values from Component 4 (such as `chunk_article_1`) to the real article IDs.

C. Adaptive retrieval depth control.

Table 5.3: Adaptive retrieval depth by detail level

`detail_level`	Retrieved content	Method

detail_level	Retrieved content	Method
brief	Headline, lead paragraph, plus 1–2 best paragraphs	Fixed fetch plus small hybrid search
structured	All paragraphs in original order, within a token budget	Fetch by article_id, sort by para_index
full	Hybrid top-k plus neighbor paragraphs (±1)	Hybrid search, then neighbor expansion
QUESTION_ANSWER	Top-k for the question	Hybrid search only

D. Prompt construction. A fixed template has four parts: (1) system rules (answer only from the given article text, say so if the answer is not there, write in Sinhala); (2) the prompt_modifier injected unchanged; (3) a short instruction for the intent; (4) the retrieved paragraphs in order with headline and date. Because the output is read aloud, extra rules forbid markdown, bullet symbols and emojis, ask for numbers that are easy to speak, and use spoken ordinals such as "පළමුව, දෙවනුව" for step-by-step style.

E. Generation. gpt-5.4-mini is called with temperature between 0 and 0.3 so the answer stays close to the article. The maximum length depends on detail_level (brief is about 2–4 sentences). A Sinhala-purity check validates the output, with one retry if it fails.

5.2.4 Security and privacy implementation

- **Secrets:** the API key is stored in an environment variable and excluded from Git using .gitignore.
- **Grounding:** the prompt forces answer only from retrieved article text, which reduces false information being spoken to the user.
- **Scope filtering:** the article_id metadata filter prevents leakage of unrelated content into an answer.
- **Input validation:** the request is validated against the JSON schema, and unknown intents or style values fall back to safe defaults.

- **Data use:** the newspaper corpus is used only for research, and no user study is reported in this thesis (Section 6.3).

5.3 Integration with the Other Components

Component 2 sends the corrected text and Component 4 sends the personalization JSON. Component 3 exposes one endpoint that accepts the agreed schema and returns the output JSON to the voice module. The field `chunk_ids_used` is returned only as a trace for debugging and evaluation.

5.3.1 User-interface implementation

Component 3 has no visual interface, because the target users are visually impaired and the interaction is voice-based through Component 4. The "interface" of this component is therefore (a) the JSON contract and (b) the shape of the text, which is written for listening: short sentences, no symbols, and spoken-friendly numbers and ordinals.

5.4 Verification and Software Testing

This section describes how Component 3 was verified against the requirements defined in Section 4.1 and how its behavior was tested before the results in Section 6.1 were produced. Verification was organized as a layered activity, ranging from checks on individual modules to an offline evaluation of the complete pipeline. This layering was necessary because an error at an early stage of the pipeline, for example invisible characters left in the corpus, propagates into every later stage and may become visible only in the final spoken answer.

5.4.1 Verification and testing strategy

The verification effort addressed two questions. The first was whether the component had been built in accordance with the functional and non-functional requirements (Section 4.1) and with the JSON contract shared with Components 2 and 4. The second was whether the component produces answers that are faithful to the article, correct, written in Sinhala and of a length suitable for listening, which is the subject

of the evaluation reported in Section 6.1. The strategy used the quality characteristics of ISO/IEC 25010 and the test-planning guidance of the ISO/IEC/IEEE 29119 series as reference points, as stated in Section 4.5. These standards were used as guidance only, and no formal certification is claimed.

Because the development process was iterative and each increment produced a testable unit before the next increment started (Section 5.1.4), verification was carried out continuously during implementation and not only at the end.

Table 5.4: Levels of verification

Level	Scope	Main objects of verification	Where reported
Module level	Individual modules: corpus cleaner, chunkier, indexes, request resolver, scope selector, depth controller, prompt builder, generator and checker, output adapter	Correct behavior of each module on representative and edge-case inputs	Section 5.4.3
Interface level	Input and output JSON contract between Component 3 and Components 2 and 4	Schema validity, identifier mapping and unchanged propagation of the request	Sections 5.4.4 and 6.4
Pipeline level	Complete answering pipeline run on 72 labelled queries	Retrieval accuracy, faithfulness, abstention, correctness and language quality	Section 6.1
Robustness and security	Secret handling, privacy controls and input handling	Absence of leakage and safe defaults for invalid input	Section 5.4.5

5.4.2 Test environment and test data

All tests were executed on the development machine described in Section 5.1: a Lenovo 82KB laptop with an 11th Gen Intel Core i5-1135G7 processor, 8 GB of RAM and integrated graphics, running 64-bit Windows 11 and Python 3.13.0. The dense index used ChromaDB with the `intfloat/multilingual-e5-small` embedding

model, and answers were generated through the hosted `gpt-5.4-mini` service. Tests that involved generation therefore required an internet connection.

Three sets of test data were used.

- **Corpus.** The 500 Sinhala newspaper articles, after processing by the cleaning and chunking pipeline described in Section 5.2.2.
- **Personalization records.** The nine sample records supplied by Component 4, which cover the intents SUMMARIZE, EXPLAIN, SIMPLIFY and ELABORATE, the style classes *Simple*, *Detailed* and *Step-by-step*, and the detail levels *brief*, *structured* and *full*. These were extended with synthetic requests (Section 3.2.3).
- **Labelled evaluation set.** 72 queries, of which 56 are answerable, each with a gold source article, a reference answer and annotated key facts, and 16 are unanswerable (8 off-topic and 8 near-topic), as defined in Section 3.4.

To limit sources of variation, a fixed random seed and a low generation temperature (0 to 0.3) were used, the BM25 and Chroma indexes were saved to disk, indexing was idempotent, and all settings were held in a single configuration file. Under these conditions, the same data and settings produce comparable results on repeated runs, except for the residual variability of the hosted model (Section 3.7.3).

5.4.3 Module-level verification

Each module was verified against the behavior specified in Sections 4.2 and 5.2. Table 5.5 lists the properties that were checked and the requirements to which they relate. The cleaner and the chunkier were verified before the indexes were built, because invisible characters inside Sinhala words break keyword matching and weaken embeddings (Section 5.2.2), and an undetected defect at this stage would have affected every later result.

Table 5.5: Properties verified at module level

Module	Properties verified	Requirement

Module	Properties verified	Requirement
Corpus cleaner	Soft hyphens (U+00AD), zero-width spaces and surplus whitespace are removed; the zero-width joiner (U+200D) is preserved so that conjuncts such as ශ්‍රී remain intact; text is in NFC form; junk lines are removed and empty or very short articles are skipped	FR2
Chunker	Chunk 0 is the headline and chunks 1 to n are paragraphs; identifiers follow `{article_id}::p{index:03d}` and are unique; all metadata fields are present; long paragraphs are split at sentence ends; paragraphs of fewer than about 15 words are merged with a neighbor	FR3
Dense index	Every chunk is stored with its metadata; upsert keyed by the chunk identifier prevents duplicates when indexing is repeated; the `passage:` and `query:` prefixes are applied for the E5 model	FR4, NFR6
BM25 index	The index is saved to disk and reloaded at start-up; character 3 to 4-grams allow inflected forms such as සැකකරු, සැකකරුවන් and සැකකරුගේ to match	FR4
Request adapter and resolver	Required fields are validated; aliases such as `chunk_article_1` are mapped to real article identifiers and an unknown identifier is reported as an error and not guessed; the precedence rules are applied to conflicting signals, for example `style_class: Detailed` with `language_style: simple`	FR1, FR9
Scope selector	The `article_id` filter is applied for SUMMARIZE, EXPLAIN, SIMPLIFY, ELABORATE and IDENTIFY_CONTENT; question answering searches the current article first and uses the whole corpus only when the question is outside it or no article is active	FR6
Depth controller and hybrid retriever	The *brief*, *structured*, *full* and question-answering modes return the content specified in Section 5.2.3; each retriever returns its top 20 results, which are fused with RRF (k = 60); duplicates are removed and chunks are passed on in paragraph order	FR5, FR7
Prompt builder	The four-part template is assembled; `prompt_modifier` is inserted unchanged; the intent instruction is applied; voice rules forbid markdown, bullet symbols and emojis	FR8, NFR4

Module	Properties verified	Requirement
Generator and checker	Temperature is held between 0 and 0.3; maximum length follows the detail level; the Sinhala-purity check is applied with one retry	FR10, NFR3
Output adapter	The five output fields (`answer_text`, `article_id`, `chunk_ids_used`, `style_applied`, `detail_level_applied`) are returned in the agreed format	FR1, NFR5, NFR6

5.4.4 Interface and integration testing

Integration with the other components was verified at the level of the JSON contract, as reported in Section 6.4. Each test query was passed to Component 3 in the structured record format that the personalization stage produces (intent, style class, prompt modifier and personalization flags). An adapter check confirmed that the question reached the pipeline unchanged. Component 3 returned the generated answer together with the retrieved passages, their sources and the quality measures, which is the information required by Component 4 and by the evaluation. The following aspects were covered.

- **Input contract.** Records conforming to the agreed schema were accepted, and optional fields such as `detail_level` were handled by defaults when absent.
- **Identifier mapping.** Placeholder identifiers used in the sample files were mapped to real article identifiers, and an unknown identifier produced an explicit error.

This verification has an important limitation. The OCR and text-correction stages (Components 1 and 2) were bypassed, so the corrected text was assumed to be given. The records were built from the sample record and from the labels in the test set and were not produced by the live personalization stage. Integration has therefore been verified at the interface level only, and testing together with the live voice module remains future work (Section 9.4).

5.4.5 Security and privacy verification

The security and privacy controls described in Sections 3.6.2 and 5.2.4 were verified by inspection of the source code, the configuration and the run logs, using the

OWASP Top 10 for LLM applications as a guide (Section 4.5). The following checks were made.

- **Secrets.** The API key is read from an environment variable and does not appear in the source code, the configuration file, the repository (it is excluded through `.gitignore`) or the run logs.
- **Data sent to the hosted model.** Prompts contain only public article text together with the style and request text. User identifiers are not included.
- **Scope isolation.** The `article_id` filter prevents unrelated news content from entering an answer, and the `chunk_ids_used` trace allows this to be confirmed for any answer.
- **Input validation.** Requests are validated against the JSON schema, and unknown intent or style values fall back to safe defaults.
- **Instruction safety.** Article text is treated as data and not as instructions, and the system rules in the prompt restrict answers to the supplied article text.

5.5 Deployment and Operation

This section describes how Component 3 was deployed and operated during the study. Component 3 was deployed as a locally executed research prototype in the controlled, offline experimental setting defined in Section 3.1.3. It was not deployed as a production service, and the section therefore documents the deployment arrangement, the procedure for reproducing it, the way it was operated and maintained, and the steps that a production deployment would still require.

5.5.1 Deployment model and target environment

The component runs on a single machine, apart from answer generation, which uses a hosted language model. The development machine is described in Section 5.1. Its specifications are the observed configuration used in this study and are not minimum requirements for deployment. The design needs no model training and no dedicated GPU (NFR7), which is consistent with the needs of the possible deployers, namely organizations for the blind and libraries, who require low running cost and no specialized hardware (Section 4.1.1). Table 5.6 summarizes the deployed elements.

Table 5.6: Deployed elements

Element	Location	Description
Application code and configuration	Local machine	Python 3.13.0 modules for cleaning, chunking, retrieval, prompt building, generation and output; settings held in `config.yaml`
Dense index	Local machine	Persistent ChromaDB collection holding chunk embeddings and metadata
Keyword index	Local machine	`rank_bm25` index saved to disk and loaded at start-up
Embedding model	Local machine (CPU)	`intfloat/multilingual-e5-small`, used through `sentence-transformers`
Language model	Hosted service	`gpt-5.4-mini` accessed through the provider's API under the provider's terms of use; requires an internet connection
Neighboring components	Interface only	Components 2 and 4 communicate with Component 3 through the agreed JSON contract

5.5.2 Installation and configuration procedure

The deployment can be reproduced from the version-controlled repository by the procedure in Table 5.7. The procedure follows the order of the experimental process in Section 3.3.1, because each step depends on the output of the one before it.

Table 5.7: Installation and configuration procedure

Step	Action	Result
1	Obtain the source code from the repository and check out the tagged version used for the thesis results	Code, prompt templates and the shared JSON schema file
2	Create a Python environment and install the dependencies pinned in `requirements.txt`	Library versions identical to those used in the study
3	Provide the API key through an environment variable and review `config.yaml`	Secrets kept outside the source code; parameters set for chunk limits, top-k values, the RRF constant (k = 60), generation temperature and output length per detail level

Step	Action	Result
4	Run the cleaning script on the collected articles and record the SHA-256 hash of the cleaned corpus	Cleaned articles with headline, section and date, and a recorded data version
5	Run the build script to chunk the articles and create both indexes	ChromaDB collection and saved BM25 index
6	Send a sample personalization record through the developer interface and inspect the answer, the applied style and detail level, and the chunk identifiers	Confirmation that the installation works
7	Start Component 3 so that it accepts requests in the agreed schema	Component ready to receive requests from Component 4

The raw newspaper articles are not redistributed because of copyright. A person reproducing the deployment must therefore rebuild the corpus from the sources listed in the repository, using the same cleaning script, and the built indexes are regenerated from the cleaned corpus by a single script (Section 3.7.1).

5.5.3 Runtime operation

At start-up, the component loads its configuration, opens the Chroma collection, loads the saved BM25 index and loads the embedding model. During operation it exposes one endpoint that accepts the agreed schema and returns the output JSON to the voice module (Section 5.3). Each request follows the flow described in Section 4.2.3: the input is validated and identifiers are mapped; the resolver fixes the detail level and style; the scope selector sets the search filter; the depth controller and the hybrid retriever select the chunks; the prompt builder and the generator produce the answer; the Sinhala-purity and length checks are applied; and the five output fields are returned.

Cleaning, chunking and indexing are performed once and offline, which keeps the answering path short (Section 4.2.4). Retrieval and prompt construction run locally on the CPU, whereas the generation step depends on the hosted service, so the availability of answers depends on internet access and on the availability of that service. Approximate stage timings recorded during development and reported in the group report (Table 6.3) were about 100 to 200 ms for retrieval, about 50 ms for prompt construction and about 2 to 4 s for answer generation. These indicate that the

answer is produced within a few seconds, consistent with NFR2, but they are not a controlled measurement over the 72 evaluation queries. For development and evaluation, a command-line or notebook interface allows the researcher to send a request record and to view the answer, the style and detail level applied and the chunk identifiers used. This interface is not part of the product used by the end user (Section 4.4.1).

5.5.4 Configuration, secrets and version management

- **Configuration.** All adjustable settings are held in one `config.yaml` file, so that none is hidden in the code. These are the embedding model, the chunking limits, the retrieval parameters (top 20 per retriever and RRF constant 60), the depth modes, the maximum length, the random seed, and the style and length rules.
- **Secrets.** The API key is supplied through an environment variable, excluded from Git and kept out of the logs.
- **Dependencies.** Library versions are pinned in `requirements.txt`. The name and version of the generation model, the embedding model and the dates of the generation runs are recorded.
- **Releases.** The version used to produce the thesis results is marked with a Git tag, and a SHA-256 hash identifies the cleaned corpus.
- **Interface schema.** The JSON contract with Components 2 and 4 is kept in one shared schema file under version control, so that any change to it is tracked.

5.5.5 Deployment limitations and requirements for production use

The present deployment is a research prototype. Concurrent use, response-time targets, availability monitoring, the cost per answered query and behavior on a collection much larger than about 500 articles were not evaluated in this study (Sections 8.3.2 and 8.4.1). A production service would additionally require the following.

- A financial plan for the cost of the hosted model, or a tested local alternative, because the availability, price and behavior of the hosted model can change (Sections 8.3.2 and 9.4).

- A review of the provider's data-use and retention terms, because the article text and the request text leave the local machine during generation (Section 3.6.2).
- Agreements with the newspaper publishers for any commercial use of the articles (Section 8.4.2).

Integration with, and testing against, the live OCR, text-correction and voice components, followed by validation with visually impaired listeners (Sections 6.3 and 9.4).

6 RESULTS AND VALIDATION

6.1 Research Results

The component was evaluated offline on 72 test queries: 56 answerable from the knowledge base and 16 deliberately unanswerable (8 off-topic and 8 near-topic). Each query was run once without a pre-selected passage, so that the retriever was tested on its own.

6.1.1 Retrieval (RQ1)

Table 6.1 reports retrieval accuracy at the article level and at the evidence level, where evidence means a retrieved passage from the correct article that contains the key fact.

Table 6.1: Retrieval accuracy at article level and evidence level

Metric	Source Article	Evidence passage
Hit@1	58.9%	39.3%
Hit@3	80.4%	69.6%
Hit@5	82.1%	75.0%
MRR	0.69	0.53

The correct article was found in the top five for 82.1% of queries, and a passage containing the key fact for 75.0%. The difference is largest at rank 1 (58.9% against 39.3%). This shows that the retriever often locates the correct article but ranks another passage from it above the one that holds the answer.

6.1.2 Faithfulness and abstention (RQ2)

Of the 41 answerable queries that were answered, 90.2% were judged faithful to the retrieved evidence. The system correctly declined 15 of the 16 unanswerable queries (93.8%): all 8 off-topic queries (100%) and 7 of the 8 near-topic queries (87.5%). The cost of this cautious behavior was a false refusal rate of 26.8% on answerable queries.

6.1.3 Correctness by intent type (RQ3)

The mean correctness score was 1.34 on the 0 to 2 scale (about 67% of the maximum), and 53.6% of answers contained the annotated key fact. Table 6.2 shows the scores by intent type.

Table 6.2: Mean correctness by intent type

Intent Type	Mean correctness
Summarization	1.64
Elaboration	1.46
Direct answer	1.21
Simplification	1.20

Summarization and elaboration scored higher than direct-answer and simplification queries. Direct-answer questions ask for a single fact, so a retrieval miss on that fact lowers the score directly.

6.1.4 Language quality and format (RQ4)

Table 6.3: Language quality and format results

Metric	**Result**
Mean Sinhala script purity	0.986
Answers within the word limit	95%
Mean perplexity (answered queries)	1.143 (median 1.138)
Perplexity, correct vs incorrect answers	1.143 vs 1.141

The answers were almost entirely in Sinhala script. Perplexity was low, but it was almost identical for correct and incorrect answers. It therefore indicates consistency of wording and does not distinguish correct answers from incorrect ones.

6.2 Product and Technical Results

At the product level, the results in Section 6.1 show that Component 3 delivers the behaviors required of it in Section 4: answers that stay within the retrieved newspaper text, a refusal when the text does not contain the answer, output in Sinhala, and answer length within the set limit. Table 6.4 links each behavior to its supporting evidence.

Table 6.4: Required behaviors of Component 3 and supporting evidence

Behavior	Evidence
Answers grounded in retrieved text	90.2% faithfulness
Declines question the knowledge base cannot answer	93.8% correct abstention (15 of 16)
Output in Sinhala	Mean purity 0.986
Length control for listening	95% within word limit
Support for four intent types	Correctness reported for all four (Table 6.2)

6.3 User or Stakeholder Validation

The validation reported in this Section is based on automated measures. Direct validation with visually impaired listeners is not part of the results reported here. The needs identified for the stakeholders in Section 4.1.1 were instead checked through measurable proxies that matter to a listener who cannot verify the printed page: faithfulness to the article (90.2%), refusal when information is missing (93.8%, 15 of 16 unanswerable queries), pure Sinhala output (0.986), and answers within a length suited to listening (95%). These measures indicate that the system behaves as intended for such users.

6.4 Integration Results

Integration was validated at the interface level. Each test query was passed to Component 3 in the structured record format that the personalization stage produces (intent, style class, prompt modifier and personalization flags), and an adapter check confirmed that the question reached the pipeline unchanged. Component 3 returned the generated answer together with the retrieved passages, their sources and the quality measures, which is the information that Component 4 and the evaluation need. The OCR stage (Components 1 and 2) was bypassed, so the corrected text was assumed to be given. The records were built from the sample record and the labels in the test set, and they were not produced by the live personalization stage.

6.5 Summary of Findings

The evaluation shows that the RAG component is strong in grounding and language quality and limited mainly by passage-level retrieval and cautious abstention.

- **RQ1 (retrieval):** The correct article was in the top five for 82.1% of queries and an evidence passage for 75.0%, but the evidence passage was ranked first for only 39.3%.
- **RQ2 (faithfulness and abstention):** 90.2% of answers were faithful, and 15 of 16 unanswerable queries (93.8%) were declined (all 8 off-topic queries and 7 of the 8 near-topic queries, 87.5%), at the cost of a 26.8% false refusal rate.
- **RQ3 (intent types):** Correctness was highest for summarization (1.64) and elaboration (1.46) and lower for direct-answer (1.21) and simplification (1.20) queries.
- **RQ4 (language and format):** Script purity was 0.986, 95% of answers were within the word limit, and perplexity was low (mean 1.143) without being linked to correctness.

These findings come from a small test set, a single run and an LLM-based judge, so they are best read as indicative of the system's behavior in this setting.

7 DISCUSSION

7.1 Interpretation of Findings

The results in Section 6 show a component that is reliable in what it says and less reliable in finding what is needed to say it. Three patterns stand out.

Retrieval. The correct article was found in the top five for 82.1% of answerable queries, but the evidence passage was ranked first for only 39.3%. This indicates that the retriever usually reaches the right article and then ranks a neighboring passage above the one holding the answer. The false refusal rate (26.8%) is similar size to the share of queries whose evidence was not retrieved in the top five (25.0%). This suggests, although this evaluation does not prove it, that many refusals follow retrieval misses and not generation errors. A per-question check of the refused queries would confirm this. The rule-based key-fact rate (53.6%) was also lower than the judged correctness (about 67% of the maximum). Exact string matching of Sinhala key facts is therefore stricter than the judge, and the evidence-level figures may slightly underestimate true retrieval success.

Grounding and caution work together. Faithfulness was 90.2%, and 15 of the 16 unanswerable queries (93.8%) declined, including all 8 off-topic queries and 7 of the 8 near-topic ones (87.5%). Near-topic queries are the harder test because the knowledge base contains related text, and the single near-topic failure shows where the risk lies. These results are consistent with the design decision to restrict answers to retrieved article text. The price is cautious behavior, which leaves some answerable questions unanswered. For a listener who cannot check the page, a refusal is a safer error than an invented answer, but it still reduces usefulness.

Correctness depends on the request type. Summarization (1.64) and elaboration (1.46) scored higher than direct-answer (1.21) and simplification (1.20) queries. A plausible explanation is that direct-answer questions depend on one fact, so a single retrieval miss makes the answer wrong, whereas summaries and elaborations draw on several paragraphs and tolerate a missed one. The lower simplification score may

reflect the trade-off between simple wording and completeness, but this evaluation does not test that explanation.

Language quality does not guarantee correctness. Sinhala purity was high (0.986) and 95% of answers were within the length limits set for listening. Perplexity was almost identical for correct and incorrect answers (1.143 and 1.141). A fluent answer is therefore not evidence of a correct one, which matters for listeners who rely on how an answer sounds.

7.2 Comparison with Literature Baselines and Existing Solutions

This section compares the results of Component 3 with the findings in the literature reviewed in Section 2. The comparison is qualitative. The reviewed sources report results on different languages, tasks, benchmarks and models, and no study of retrieval-augmented generation (RAG) for Sinhala newspaper question answering was found in those sources (Section 2.5.1). Numerical values from the literature are therefore not set against the figures in Section 6, because the two would not be measured on the same basis. In addition, this evaluation did not compare the design with alternatives such as dense-only retrieval, BM25-only retrieval or prompt-only personalization (Section 1.7), so the section does not claim that the design outperforms any of them.

Retrieval. Dense passage retrieval has been reported to outperform a strong BM25 system by 9% to 19% absolute in top 20 passage accuracy on open-domain question answering in English [9], and Reciprocal Rank Fusion (RRF) has been reported to outperform Condorcet fusion and individual rank-learning methods [11]. These findings are the reason for combining the two ranking methods through RRF in Component 3. The present results are consistent with a working hybrid retriever, because the correct article was in the top five for 82.1% of answerable queries and an evidence passage for 75.0%, using the multilingual E5 embeddings [8] with BM25 [10]. However, this evaluation did not run dense-only or BM25-only retrieval, so it cannot show how much each part, or the fusion step, contributed on Sinhala news. The weaker result at rank 1 (39.3% at evidence level) shows that the benefit reported in English open-domain settings [9], [11] should not be assumed to carry over fully

to Sinhala newspaper text, where inflected word forms may weaken keyword matching (Section 1.4).

Faithfulness and abstention. Surveys of hallucination describe generated content that is unsupported by the source as a documented problem across generation tasks [7]. A benchmark of RAG in English and Chinese found that language models still have clear weaknesses in noise robustness and negative rejection, that is, declining a question when the supplied documents do not contain the answer [12]. In Component 3, 90.2% of answered queries were judged faithful and 15 of 16 unanswerable queries (93.8%) were declined. These results suggest that restricting the answer to retrieved article text, together with an instruction to decline, is an effective way to limit unsupported answers in this setting, in line with the purpose of RAG [5], [6]. They cannot be compared directly with the results in [12], because the languages, test queries and models differ, the faithfulness judgement here was made by a language-model judge, and only 16 unanswerable queries were tested. The 26.8% false refusal rate also shows that the behavior reported in [12] appears here in the opposite form: the system declines some questions that the article can answer.

Use of retrieved context. Language models use relevant information less reliably when it is placed in the middle of a long input [13]. The finding that the correct article is usually found but the evidence passage is often not ranked first is consistent with this concern for the quality of the context supplied to the model, and with the design choice to control how much article content is retrieved. This evaluation did not vary the amount or order of the context, so it does not test the finding in [13] directly.

Sinhala language technology. The Sinhala studies reviewed describe a language with limited resources [2], [3] and report work on foundational tasks such as text classification with pre-trained models [4]. The present results have evidence of a different kind. They show that a prompt-based approach with retrieval and a hosted model can produce answers that are almost entirely in Sinhala script (mean purity 0.986) and within the length limit set for listening (95%) without any model training. No Sinhala RAG baseline for newspaper question answering was found in the

sources reviewed, so these figures serve as a first reference point for later work, not as a comparison against an earlier result.

Existing solutions. The sources reviewed do not describe existing reading aids for printed Sinhala newspapers (Section 2.4.1), and no product-level comparison is therefore made. Of the four gaps identified in Section 2.5.1, the results speak most directly to language coverage and evaluation for listening, because Section 6 reports a labelled evaluation of faithfulness, abstention, script purity and length for Sinhala.

7.3 Research Contribution

The research contributes the following.

1. **A style-aware hybrid RAG design for Sinhala newspaper reading.** The design uses a structured control record to adjust both the prompt and the amount of article content retrieved and combines dense and BM25 retrieval through Reciprocal Rank Fusion within paragraph-level chunks (Sections 4.2 and 4.3).
2. **An evaluation framework for Sinhala newspaper question answering.** It consists of a labelled test set of 72 questions, including unanswerable questions of two kinds (off-topic and near-topic), and measures that cover retrieval, faithfulness, abstention, correctness by intent, and language format (Section 3.4).

These contributions describe the design and its measured behavior. The evaluation did not compare the design with alternatives (for example, dense search only, or prompt-only personalization), so this thesis does not claim that the design outperforms them.

7.4 Product Contribution

The product contribution is a working Component 3 that can be integrated into the four-component reading system. It accepts the personalization record, answers from the article text, returns speech-ready Sinhala, and returns the identifiers of the passages used, which allows every answer to be traced. The results support several of

the design requirements: grounded answers (NFR1), Sinhala output (NFR3 and FR10), length control for listening, and refusal when information is missing. The design needs no model training and runs on ordinary hardware with a hosted language model (NFR7). Its interaction with the other components is limited to a fixed JSON contract (NFR5). The evaluation runner is a reusable artefact that repeats the same measurements whenever the component changes. Response time, user acceptance and full integration with the voice module were not measured in this study.

7.5 Limitations and Threats to Validity

Construct validity

- Perplexity is only a proxy for fluency, and the findings show that it does not reflect correctness.
- Listener needs were checked through proxy measures and not with listeners (Section 6.3).

Internal validity

- Faithfulness and correctness were scored by a language-model judge. Such a judge can be inconsistent and may share biases with the generating model. Agreement with a manual sample should be reported as described in Section 3.3.2.
- Key-fact matching uses exact strings and may miss valid variants of inflected Sinhala words.
- Each query was run once on a hosted model, and the output may differ between runs.

External validity

- The knowledge base contains about 500 articles, and the test set contains 72 questions. Results may not be carried over to other newspapers, topics or larger collections.

Conclusion validity

- With 56 answerable questions, one question equals about 1.8 percentage points. Differences such as those between intent types are indicative only, and no confidence intervals or significance tests were computed.
- Only 8 near-topic questions were tested, so the near-topic abstention rate rests on a very small sample.

Scope of the evaluation

- The OCR stage and Component 4 were not part of the test (Section 6.4).

7.6 Implications

For assistive reading tools. Grounding and abstention are core requirements and not optional extras, because a listener cannot verify an answer. Systems of this kind should report faithfulness and refusal behavior alongside accuracy.

For retrieval design. The gap between article-level and passage-level retrieval shows where effort should go first. Improvements to passage ranking, such as re-ranking or adding neighboring paragraphs, are likely to reduce false refusals without weakening grounding.

For Sinhala language technology. The results show that a prompt-based, retrieval-supported approach with a hosted model can produce well-formed Sinhala answers from newspaper text without model training. Retrieval over a morphologically rich language remains the weaker part.

For evaluation practice. Fluency measures such as perplexity should not be used alone to judge answer quality, because they do not separate correct from incorrect answers.

8 COMMERCIALIZATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

8.1.1 Target users

The primary users are visually impaired readers who want to follow Sinhala newspapers by listening and who need to ask questions about an article (Section 4.1.1). Secondary users are caregivers and family members who help set up the system. Possible deployers are organizations for the blind and libraries, who need low running cost, Sinhala-language support and no specialized hardware.

8.1.2 Value proposition

The component gives listener control over how much is heard and in what wording, and it answers from the article being read. Four features support this value:

- **Grounded answers.** Answers are restricted to the retrieved article text, which matters because a listener cannot check the printed page. Faithfulness was 90.2% in the evaluation (Section 6.1.2).
- **Honest gaps.** The system declines questions that the article does not answer (93.8% correct abstention: 15 of 16 unanswerable queries declined), which is safer for a listener than an invented answer.
- **Personalization.** Wording style and level of detail follow the user's request.
- **Speech-ready Sinhala output.** Answers are written without visual symbols and in a form a speech engine can read.

These benefits have been shown by automated measures only. Whether listeners find the answers useful has not been tested (Section 6.3).

8.2 Market and Competitive Position

8.2.1 Alternatives available to users

A listener who wants access to printed news has three kinds of alternative: general-purpose AI assistants that answer questions from broad model knowledge, screen readers and text-to-speech tools that read text aloud without answering questions about it, and human readers such as family members or volunteers. These

alternatives were not benchmarked in this study, and current market figures and product details should be taken from the literature review.

8.2.2 Competitive position

The component is positioned in the gap between reading text aloud and general-purpose question answering. Three points distinguish it:

- It answers from the article being read and scopes retrieval to that article, whereas general-purpose assistants may draw on the model's memory or other sources.
- It is designed for Sinhala newspaper text, including hybrid retrieval to handle personal names and inflected word forms.
- It is one component of a complete chain from a scanned page to a spoken answer, with a fixed interface to the other components.

These are design differences. The evaluation did not compare the component with other products, so no performance advantage is claimed.

8.3 Business or Sustainability Model

8.3.1 Possible models

No business model was tested in this study. The following options follow the needs of the stakeholders:

- **Institutional provision.** Organizations for the blind, libraries or public bodies deploy the system and make it free for the end user. This suits the stakeholder needs in Section 4.1.1 and keeps the service accessible.
- **Partnership with publishers.** Newspaper publishers provide licensed article access, and the service helps them reach readers who cannot read print.
- **Grant or non-profit funding.** Funding accessibility or digital inclusion programmes cover development and hosting.

A model that depends on the user paying is less suitable, because the target group may have limited income.

8.3.2 Costs and sustainability

The design needs no model training or specialized hardware (Section 4.1.3), which lowers the starting cost. The main running costs are the hosted language model, hosting of the indexes, and maintenance. The cost per answered query was not measured in this study. The service also depends on a hosted model whose availability, price and behavior can change, so a sustainable deployment would need either a financial plan for this cost or a tested local alternative (Section 9.4). Keeping the knowledge base current would also need continued access to new newspaper articles.

8.4 Scalability, Intellectual Property and Compliance

8.4.1 Scalability

The design supports growth in the number of articles because new chunks can be added to the indexes without rebuilding the whole system, and indexing is repeatable (Section 4.1.3). It was evaluated on about 500 articles, so behavior on a much larger collection is unknown. Retrieval accuracy and response time could change as the collection grows, and response time was only approximated from stage timings (Section 5.5.3). Scaling to more users would also increase the number of calls to the hosted model.

8.4.2 Intellectual property

- **Newspaper content.** The articles are copyrighted. In this study they were used only for non-commercial academic research, cited by source and date, and not republished (Section 3.6.1). Any commercial use would need agreements with the publishers.
- **Software and data.** The source code, configuration and evaluation scripts are kept in a version-controlled repository (Section 3.7.1). The license under which they could be shared or sold remains to be decided and would need the agreement of the project team and the institution.
- **Third-party components.** The hosted language model, embedding model and open-source libraries are used under their own terms, and these terms

would need to be checked before commercial use. The published work that informed the retrieval design is acknowledged in the thesis.

8.4.3 Compliance

A deployed service would need to follow the privacy and AI-governance practices described in Section 4.5. Spoken requests and answers are not logged, which follows data minimization under Sri Lanka's Personal Data Protection Act of 2022. Article text is sent to the hosted model, so the provider's data-use terms would need to be reviewed. The system has not been formally certified against any standard. It is intended for reading news and is not suitable for medical, legal or financial advice.

8.5 Contribution to the Sustainable Development Goals

The work is most closely linked to the following goals. The links describe the intended contribution and not a measured outcome.

- **SDG 10: Reduced Inequalities.** The system aims to give blind and low-vision readers independent access to printed Sinhala news, which reduces a barrier faced by people with disabilities.
- **SDG 16: Peace, Justice and Strong Institutions (target 16.10, public access to information).** Access to news supports informed participation in public life.
- **SDG 4: Quality Education.** Listening to and asking questions about written material supports learning and literacy for people who cannot read print.
- **SDG 9: Industry, Innovation and Infrastructure.** The work adds to language technology for Sinhala, a low-resource language, using an approach that needs no model training.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis addressed the problem of giving visually impaired readers reliable, personalizing answers about printed Sinhala newspaper articles. A listener cannot check a spoken answer against the page, so the answer must come from the article being read and must be honest about what the article does not contain. Sinhala is a low-resource language, which limits the use of ready-made models and evaluation resources.

The results answer the four research questions as follows.

- **RQ1 (retrieval).** The correct article appeared in the top five for 82.1% of answerable queries, and a passage containing the key fact for 75.0%. The evidence passage was ranked first for 39.3%, so passage-level ranking is weaker than article-level retrieval.
- **RQ2 (faithfulness and abstention).** 90.2% of answered queries were judged faithful to the retrieved text, and 15 of 16 unanswerable queries (93.8%) were declined. The cost was a false refusal rate of 26.8% on answerable queries.
- **RQ3 (intent types).** Mean correctness was 1.64 for summarization, 1.46 for elaboration, 1.21 for direct answers and 1.20 for simplification, on a 0 to 2 scale.
- **RQ4 (language and format).** Mean Sinhala script purity was 0.986, and 95% of answers were within the length limits set for listening. Perplexity was low but did not separate correctly from incorrect answers.

The overall conclusion is that the design is effective at keeping answers grounded, in Sinhala and within the intended length. Its main limitation is finding the exact passage that holds the answer.

9.2 Achievement of Objectives

The objectives of this study are assessed below against the evidence reported in Section 6 and the limitations stated in Section 7.5. Each objective is judged as achieved, largely achieved, partially achieved or not achieved.

The first objective was to design a style-aware RAG component that works with the other components of the reading system through a fixed JSON contract. This objective was achieved. The design in Section 4 defines the input and output records, the rules for resolving conflict signals and the modules of the pipeline, and the integration check in Section 6.4 confirmed that requests in the contract format reached the pipeline and returned the answer together with the passages used. This was validated at the interface level only, because the OCR stage was bypassed and the records were built from the sample record and not produced by the live personalization stage.

The second objective was to build hybrid retrieval over a cleaned Sinhala newspaper corpus, combining dense and BM25 search through Reciprocal Rank Fusion on paragraph-level chunks. The retrieval pipeline was built and evaluated, but its accuracy at passage level was limited, so the objective was achieved with this reservation. The correct article appeared in the top five for 82.1% of answerable queries and a passage containing the key fact for 75.0%, but the evidence passage was ranked first for only 39.3%. The evaluation did not compare hybrid retrieval with dense-only or BM25-only retrieval, so the benefit of the hybrid design over these alternatives has not been measured. This comparison is left to future work.

The third objective was to produce answers that are grounded in the article and to decline questions that the knowledge base cannot answer. This objective was largely achieved. Of the queries answered, 90.2% were judged faithful to the retrieved text, and 15 of 16 unanswerable queries (93.8%) were declined, including all 8 off-topic queries and 7 of the 8 near-topic queries (87.5%). The limitation is that the system also declined 26.8% of answerable queries, and about one in ten answered queries contained an unsupported statement, which is a significant risk for a listener who cannot verify the answer.

The fourth objective was to produce speech-ready Sinhala answers of a length suited to listening. This objective was largely achieved. Mean Sinhala script purity was 0.986 and 95% of answers were within the length limits set for the request. Perplexity was low, but it did not separate correctly from incorrect answers, so fluency was not taken as evidence of correctness. Speech quality itself, as heard by a listener, was not evaluated.

The fifth objective was to support different request types and wording styles. This objective was partially achieved. Correctness was measured for four intent types and ranged from 1.64 for summarization to 1.20 for simplification on a 0 to 2 scale. However, the ablation of retrieval depth control and the Step-by-step style are not reported in Section 6, so the personalization features of the design have not been fully evaluated. Their comparison with prompt-only personalization is left to future work.

In summary, the objective that concerns the full range of personalization remains partly open. Validation with listeners and within the complete four-component system was not part of this study and is addressed in future directions in Section 9.4. The research gaps stated in Section 1.4 were identified and an evaluation framework was built for them, but comparisons with dense-only, BM25-only and prompt-only alternatives were not made and are left to future work.

9.3 Individual Research and Product Contributions

Research contributions.

- A style-aware hybrid RAG design for Sinhala newspaper reading, in which a structured control record adjusts both the prompt and the amount of article content retrieved.
- A labelled evaluation set and measurement framework for Sinhala newspaper question answering, covering retrieval, faithfulness, abstention (off-topic and near-topic), correctness by intent and language format.
- A baseline set of measured results for this design.

Product contributions.

- A working Component 3 that accepts the personalization record, answers from the article text, returns speech-ready Sinhala and reports the passages used, so that every answer can be traced.
- A modular design that needs no model training, runs on ordinary hardware with a hosted language model, and communicates with other components only through a JSON contract.
- A reusable evaluation runner that repeats the measurements after any change to the component.

9.4 Future Directions

The directions below follow from the limitations in Section 7.5.

Improving retrieval and abstention.

- Improve passage-level ranking, for example by re-ranking the fused results, adding neighboring paragraphs, and tuning chunk size, the number of candidates and the fusion constant.
- Examine how many of the false refusals follow retrieval misses, and calibrate when the system declines, so that unnecessary refusals fall without weakening grounding.
- Compare the design with dense-only, BM25-only and prompt-only alternatives, and run an ablation of the prompt modifier only, retrieval depth only, and both together, to test whether hybrid retrieval and style-aware retrieval depth help for Sinhala news.

Wider coverage and deployment.

- Extend the knowledge base to more newspapers, topics and a larger number of articles.
- Study options that reduce dependence on a hosted model, such as a local model, for privacy and cost.

- Test the component together with the live OCR, text-correction and voice components, and validate it with visually impaired listeners, including a controlled measurement of response time.

REFERENCES

[1] R. R. A. Bourne *et al.*, "Trends in prevalence of blindness and distance and near vision impairment over 30 years: An analysis for the Global Burden of Disease Study," *Lancet Glob. Health*, vol. 9, no. 2, pp. e130–e143, Feb. 2021.

[2] P. Joshi, S. Santy, A. Budhiraja, K. Bali, and M. Choudhury, "The state and fate of linguistic diversity and inclusion in the NLP world," in *Proc. 58th Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Online, Jul. 2020, pp. 6282–6293.

[3] S. Ranathunga and N. de Silva, "Some languages are more equal than others: Probing deeper into the linguistic disparity in the NLP world," in *Proc. 2nd Conf. Asia-Pacific Section Assoc. Comput. Linguistics and 12th Int. Joint Conf. Natural Lang. Process. (AACL-IJCNLP)*, Online, Nov. 2022, pp. 823–848.

[4] V. Dhananjaya, P. Demotte, S. Ranathunga, and S. Jayasena, "BERTifying Sinhala – A comprehensive analysis of pre-trained language models for Sinhala text classification," in *Proc. 13th Lang. Resour. Eval. Conf. (LREC)*, Marseille, France, Jun. 2022, pp. 7377–7385.

[5] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[6] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, Art. no. 248, 2023.

[8] L. Wang *et al.*, "Multilingual E5 text embeddings: A technical report," *arXiv preprint arXiv:2402.05672*, 2024.

[9] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Online, Nov. 2020, pp. 6769–6781.

[10] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[11] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, Boston, MA, USA, Jul. 2009, pp. 758–759.

[12] J. Chen, H. Lin, X. Han, and L. Sun, “Benchmarking large language models in retrieval-augmented generation,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 16, 2024, pp. 17754–17762.

[13] N. F. Liu *et al.*, “Lost in the middle: How language models use long contexts,” *Trans. Assoc. Comput. Linguistics*, vol. 12, pp. 157–173, 2024.

GLOSSARY

Abstention – The behavior of declining to answer when the supplied text does not contain the answer. In this thesis the system returns a fixed refusal sentence.

Chroma DB – An open-source vector database used to store and search the dense embeddings of the chunks.

Correct abstention rate – The proportion of unanswerable queries for which the system returns the refusal sentence.

Correctness score – A score of 0, 1 or 2 given by the language-model judge by comparing an answer with the reference answer.

Dense retrieval – Retrieval that converts questions and passages into numerical vectors and ranks passages by similarity of meaning.

Depth mode (retrieval depth) – A setting (brief, structured or full) that decides how much of the article is retrieved for a request.

Design-science approach – A research approach in which a new artefact is built and then tested through experiments.

Embedding – A numerical vector that represents the meaning of a text so that similar texts lie close together.

Evidence passage / evidence rank – A chunk from the correct article that contains the key fact, and the position at which the first such chunk is retrieved.

Faithfulness – The degree to which every claim in an answer is supported by the retrieved text.

False refusal rate – The proportion of answerable queries that the system wrongly declines to answer.

Far query – An unanswerable query on a topic outside the knowledge base, such as a product specification or general knowledge.

Gold article / gold annotation – The correct source article and the reference labels (reference answer and key facts) assigned to a test query.

Hallucination – Generated content that contradicts the source or cannot be checked against it.

Hit@k – The proportion of queries for which the correct item appears among the top k retrieved results.

Hybrid retrieval – The combination of dense retrieval and keyword (BM25) retrieval, merged by Reciprocal Rank Fusion.

Intent – The type of request made by the listener: direct answer, summarization, simplification, elaboration, explanation or identification of content.

Knowledge base – The cleaned collection of 500 Sinhala newspaper articles from which all answers are generated.

Language-model judge (LLM-as-judge) – A language model used offline to score faithfulness and correctness. It is not part of the runtime pipeline.

Length compliance (word-limit compliance) – The proportion of answers whose length is within the maximum set for the request.

Low-resource language – A language with few labelled datasets, tools and models available for language technology.

Perplexity – A measure of how predictable a text is to a language model. Here it is used only as an indicator of fluency and consistency of wording, not of correctness.

Personalization – The adaptation of the retrieved content and the wording of an answer to the needs of the listener.

Prompt modifier – A style instruction from the personalization record that is inserted unchanged into the generation prompt.

Retrieval-augmented generation (RAG) pipeline – The system that retrieves relevant passages and gives them to a language model, which generates the answer from them.

Script purity (Sinhala purity) – The fraction of an answer that is written in Sinhala script. Answers below the threshold are regenerated once.

Sparse retrieval – Retrieval based on exact term matches, here BM25.

Style class – The wording style requested for an answer: Simple, Detailed or Step-by-step.

Unanswerable query – A test question that the knowledge base cannot answer and that the system is expected to decline. It is either far or near.

Zero-width joiner (U+200D) – An invisible Unicode character that is kept during cleaning because Sinhala conjuncts depend on it.

APPENDICES

Appendix A Research Instruments or Extended Results

A.1 Evaluation test set

Item	Detail
Total queries	72, each run once without a pre-selected passage
Answerable	56 (direct answer, summarization, simplification, elaboration), each with a gold article, a reference answer and key fact(s)
Unanswerable	16: 8 far (off domain) and 8 near (topic in the corpus, specific fact absent)
Question source	Generated with a language model, then checked by hand for correctness and answerability
Input per query	Question, intent, style class, prompt modifier and personalization flags

A.2 Prompt template and settings

The prompt has four parts: (1) system rules (answer only from the supplied article, say so if the answer is absent, write in Sinhala); (2) the prompt_modifier, inserted unchanged; (3) a short instruction for the intent; (4) the ordered context with headline and date. It forbids markdown, bullets and emojis, and StepByStep answers use spoken ordinal words. Settings: gpt-5.4-mini, temperature 0 to 0.3, RRF k = 60, top 20 per retriever, Sinhala-purity threshold 0.85 with one retry.

A.3 Scoring instruments

Measure	Scoring rule
Faithfulness (LLM judge, temperature 0)	1 if every claim is supported by the retrieved context, otherwise 0. Answered queries only
Correctness (same judge)	2 = key facts fully agree, 1 = partly correct or incomplete, 0 = wrong, off-topic or missing. A false refusal scores 0
Key-fact rate	Rule-based check that the annotated key fact appears (cross-check of the judge)
Correct abstention	Fixed refusal sentence returned for an unanswerable query
Sinhala purity	Fraction of the answer in Sinhala script
Perplexity	From gpt-5.4-mini token log-probabilities, answered queries only

A.4 Extended results

Metric	Result
Hit@1 / @3 / @5 (source article)	58.9% / 80.4% / 82.1%
Hit@1 / @3 / @5 (evidence passage)	39.3% / 69.6% / 75.0%
MRR (article / evidence)	0.69 / 0.53
Answerable queries answered	41 of 56 (false refusal rate 26.8%)
Faithfulness in answering queries	90.2%
Abstention: far / near queries	100% / 87.5%

Metric	Result
Mean correctness (0 to 2)	1.34 (Summarization 1.64, Elaboration 1.46, Direct answer 1.21, Simplification 1.20)
Key-fact rate	53.6%
Mean Sinhala purity	0.986
Answers within word limit	95%
Mean perplexity	1.143 (correct 1.143, incorrect 1.141)

Code, configuration, prompt templates, reference answers and evaluation scripts are in the repository: https://github.com/Janindu-Muthunayaka/R26-DS-002.

Appendix B Academic Integrity and AI Use Disclosure

I declare that all use of generative AI and other automated tools in this research is listed below, in line with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses and conclusions in this thesis.

Tool	Where used	Purpose	Human verification
gpt-5.4-mini (OpenAI API)	Runtime pipeline	Generates Sinhala answers from the retrieved context	Outputs measured with the metrics in Section 3.4
gpt-5.4-mini (OpenAI API)	Offline evaluation	LLM judge for faithfulness and correctness, and perplexity scoring	A sample of judged answers checked by hand
Language model	Test set creation	Drafting evaluation questions	Every question checked by hand for correctness and answerability

Only public article text and request text were sent to the external model. No user IDs or personal data were included in any prompt. API keys were kept in environment variables and never in the code or repository. AI-assisted content was reviewed and verified by me, and no AI tool was used to fabricate data, results or citations.